



AI for Software Engineers Course
Key Notes to Help Decision Makers

https://github.com/havelsan/YZ_EGITIM

Mehmet PEKMEZCİ

AI Application Fields

- AI-Enabled Development Environment
- Usage of AI projects/tools in the Target Environment (Ships, Land Based Systems, ...)



AI vs ML vs Deep Learning vs Generative AI

ARTIFICIAL INTELLIGENCE

MACHINE LEARNING

NEURAL NETWORKS

Fully Connected

RNN

LSTM

CNN

GAN

DBN

VAE

KAN

RBM

GNN

PINN

Diffusion

TRANSFORMERS

BERT
ENC

BART
ENC-DEC

GPT
DEC

LLM

AGENTIC

REGRESSION

SVM

DECISION TREE

KNN

PCA

RANDOM FOREST

CLUSTERING

Reinforcement Learn

.....

Prolog

Comp. Vision

Automation

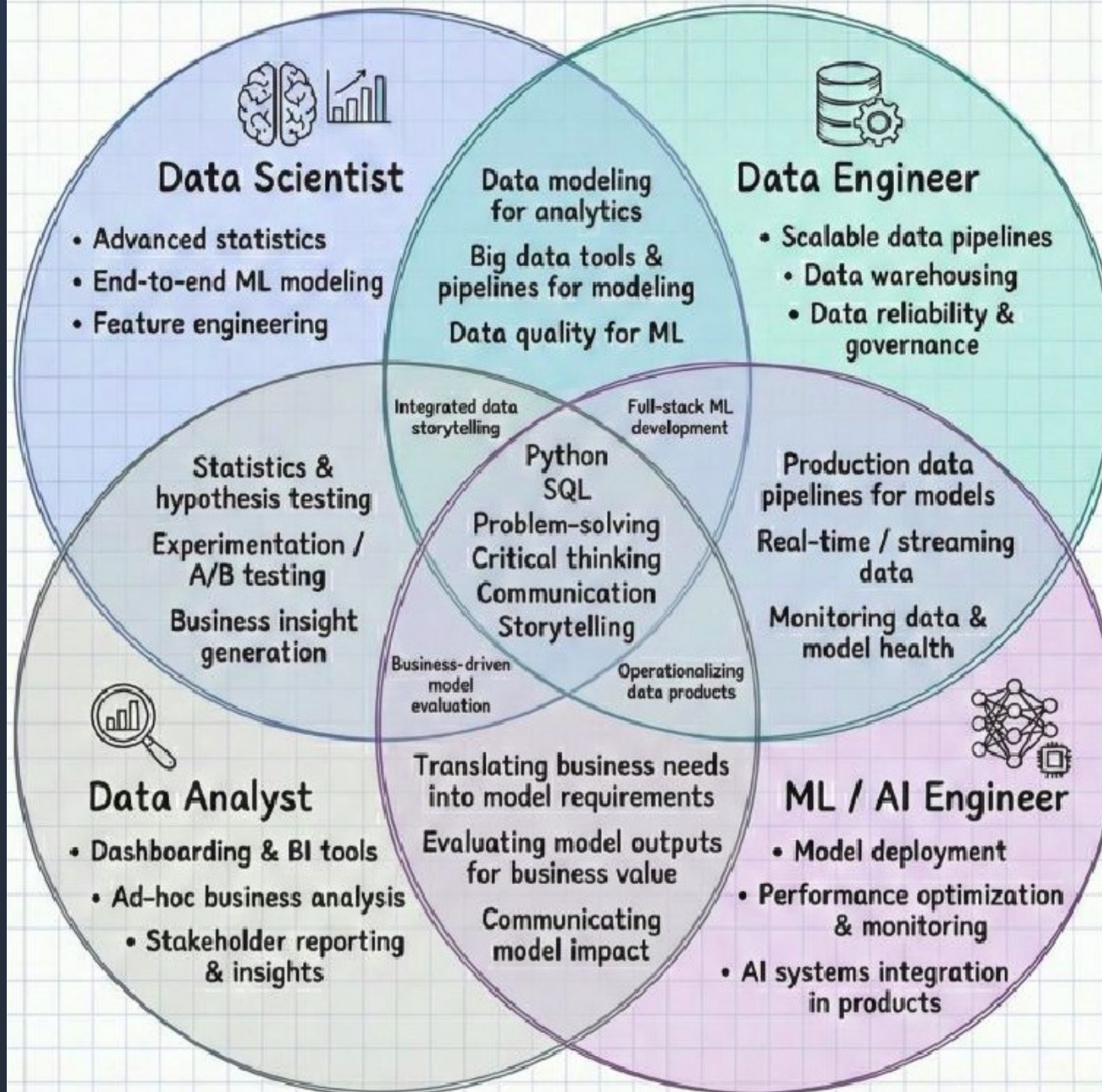
Robotics

NLP

...



Roles



Follow Jose Siles for more about Data and AI

Dataset → training → model → inference pipeline

PROBLEM DEFINITION

DATA COLLECTION

DATA CLEANING

FEATURE EXTRACTION

TRAINING

VALIDATION

TESTING

DATA VISUALIZATION

REPORT GENERATION



Dataset → training → model → inference pipeline

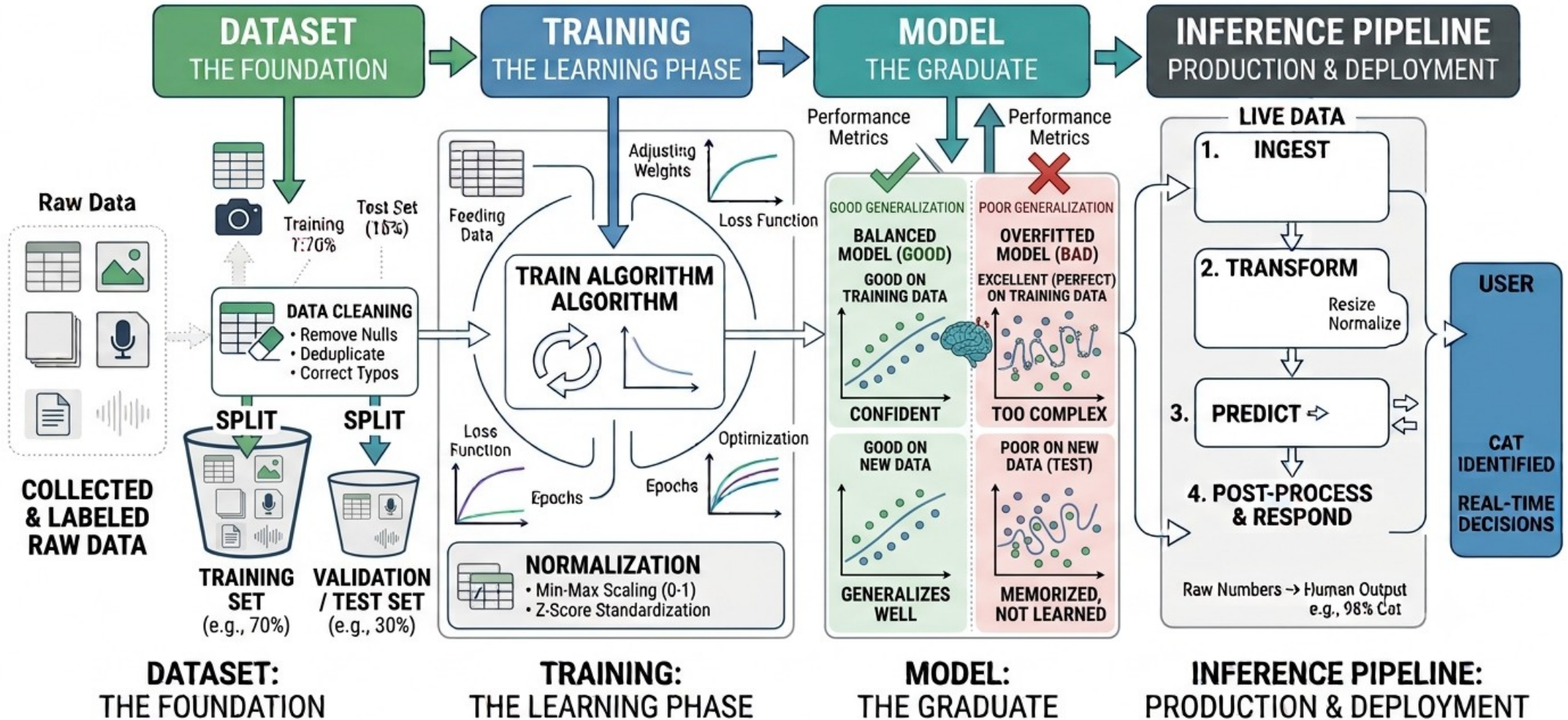
We need a lot of labeled data:

- We need to find **an automated way to collect data**
 - After building a semi automated sound recording system, I was able to collect thousands of sample data.
- We need the data to be **self labeled**.
 - Suppose that , using a monocular (single-camera) system, we try to extract the Bearing and Range of an object :
 - We record camera video,
 - We record AIS data of the objects around us, that correspond to the field of view of camera
 - We record FlightRadar etc. data also.
 - Thank to the AIS and FlightRadar data, we can say that our data is labeled (semi-labeled).



Train/test split, overfitting (intuition, not math-heavy)

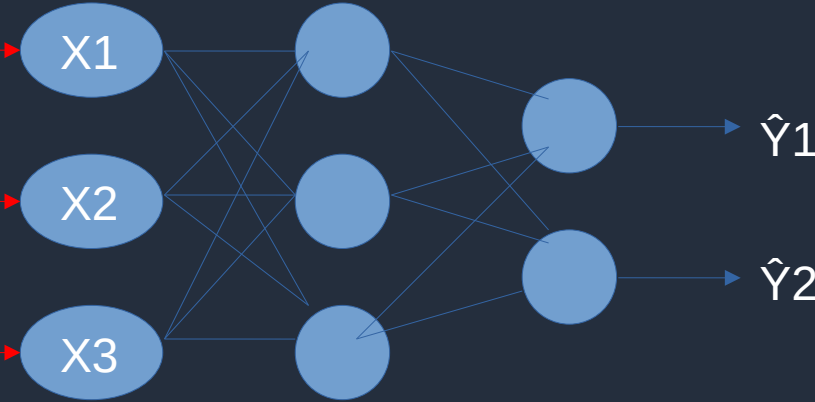
UNDERSTANDING ML PERFORMANCE & OVERFITTING



Loss Function Intuition

Loss Function = Distance Function = Metric
Loss = Error = Distance

X			Y	
X1	X2	X3	Y1	Y2
1	2	3	4	5
11	12	13	14	15
21	22	23	24	25
31	32	33	34	35
41	42	43	44	45



$$L_1 = |\hat{Y}_1 - 4| + |\hat{Y}_2 - 5|$$

$$L_2 = \sqrt{|\hat{Y}_1 - 4|^2 + |\hat{Y}_2 - 5|^2}$$

$$L = -(4 \log(\hat{Y}_1) + 5 \log(\hat{Y}_2))$$

Some of Loss Functions n = output vector length (in this case 2):

Minkowski Distance :
$$L_p = \left(\sum_{i=0}^n |Y_i - \hat{Y}_i|^p \right)^{\frac{1}{p}}$$
 , L_1 = Manhattan Distance , L_2 = Euclidean Distance

Cross Entropy :
$$L = - \sum_{i=0}^n Y_i \cdot \log(\hat{Y}_i)$$
 , Softmax , Log Loss , n=number of classes , output is one-hot encoded

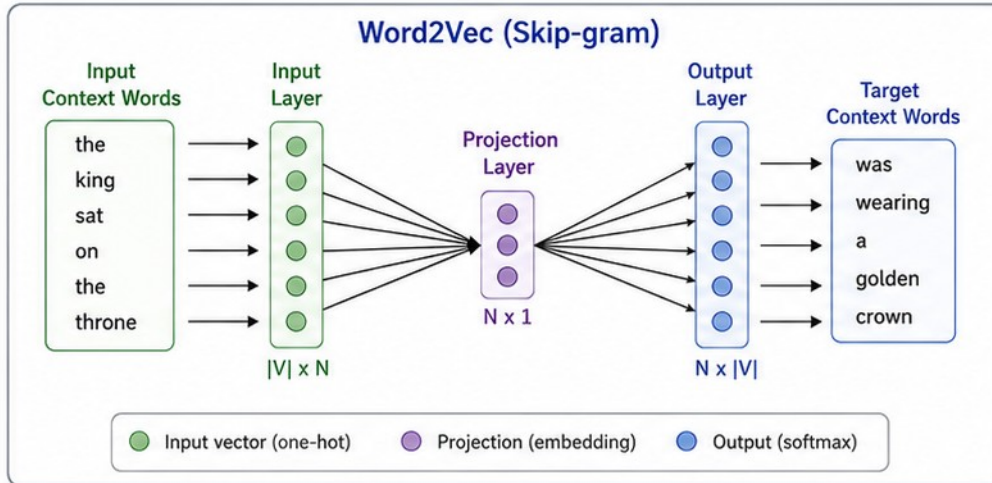


Feature vs representation learning

Word2Vec

Words are mapped to dense vectors in a continuous vector space where semantically similar words are close to each other.

Word2Vec (Skip-gram)



Each word is represented as a dense vector (embedding)

Word	v_1	v_2	v_3	...	v_n (N dimensions)
queen	0.21	-0.13	0.34	...	0.12
woman	0.19	-0.24	0.31	...	0.07
man	-0.11	0.22	-0.28	...	-0.09
king	0.22	-0.11	0.35	...	0.10

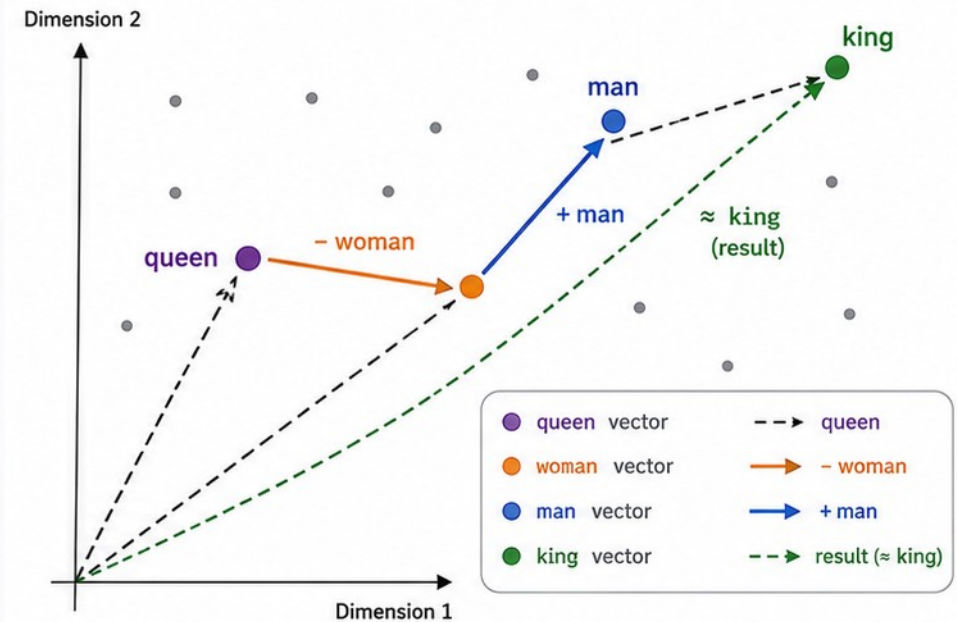
(N is typically 100–300)

Vector Arithmetic Example

$$\text{queen} - \text{woman} + \text{man} = \text{king}$$

In the vector space, the following relationship holds:

$$\vec{\text{king}} \approx \vec{\text{queen}} - \vec{\text{woman}} + \vec{\text{man}}$$



Key Idea

Word2Vec learns word embeddings so that words appearing in similar contexts have similar vectors.

This enables meaningful vector arithmetic: $\text{queen} - \text{woman} + \text{man} = \text{king}$.



ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix

Suppose we have a “machine learning model” that predicts if a track is friend or foe (IFF):

FOE=POSITIVE, FRIEND=NEGATIVE

- If the track is ACTUALLY FRIEND and the model PREDICTED as FRIEND : count as TRUE NEGATIVE
- If the track is ACTUALLY FRIEND and the model PREDICTED as FOE : count as FALSE POSITIVE
- If the track is ACTUALLY FOE and the model PREDICTED as FRIEND : count as FALSE NEGATIVE
- If the track is ACTUALLY FOE and the model PREDICTED as FOE : count as TRUE POSITIVE

Confusion Matrix For Binary Classification

	Predicted Positive	Predicted Negative
Actual Positive	Number of True Positives (TP)	Number of False Negatives (FN)
Actual Negative	Number of False Positives (FP)	Number of True Negatives (TN)



ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	True Positives (TP)	False Negatives (FN)
Actual Negative	False Positives (FP)	True Negatives (TN)

$$PRECISION = \frac{TP}{TP + FP}$$

$$RECALL = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot PRECISION \cdot RECALL}{PRECISION + RECALL}$$

$$ACCURACY = \frac{TP + FP}{ALL}$$

$$ALL = TP + FP + TN + FN$$

HIGH PRECISION : Model should not miss any FOE, but some of the FOES may be FRIEND actually.

HIGH RECALL : Model may miss some FOEs, but should be sure that no actual FRIEND may be reported as FOE.

HIGH ACCURACY : Model should not miss any FOE, and also should be sure that no actual FRIEND may be reported as FOE.

F1 :

- Completely ignores True Negatives (TN).
- This metric is used in lieu of ACCURACY, when the data is heavily imbalanced.
- Balance between catching positives and being right about them.



ML Model Evaluation : False Positive, Precision, Recall, Accuracy, Confusion Matrix

Confusion Matrix : A confusion matrix is a table used to evaluate the performance of a **classification model**.

Each X-Y-Cell Filled By :
How many of detected Class X was actually Class Y ?

Suppose our model will classify the vessels on the sea :

We suppose that it is sufficient to classify the vessels into 3 classes :

1. Naval Vessel
2. Cargo Vessel
3. Passenger Vessel

	Naval Vessel Predicted	Cargo Vessel Predicted	Passenger Vessel Predicted
Naval Vessel Actual	80	30	5
Cargo Vessel Actual	16	650	5
Passenger Vessel Actual	4	20	190

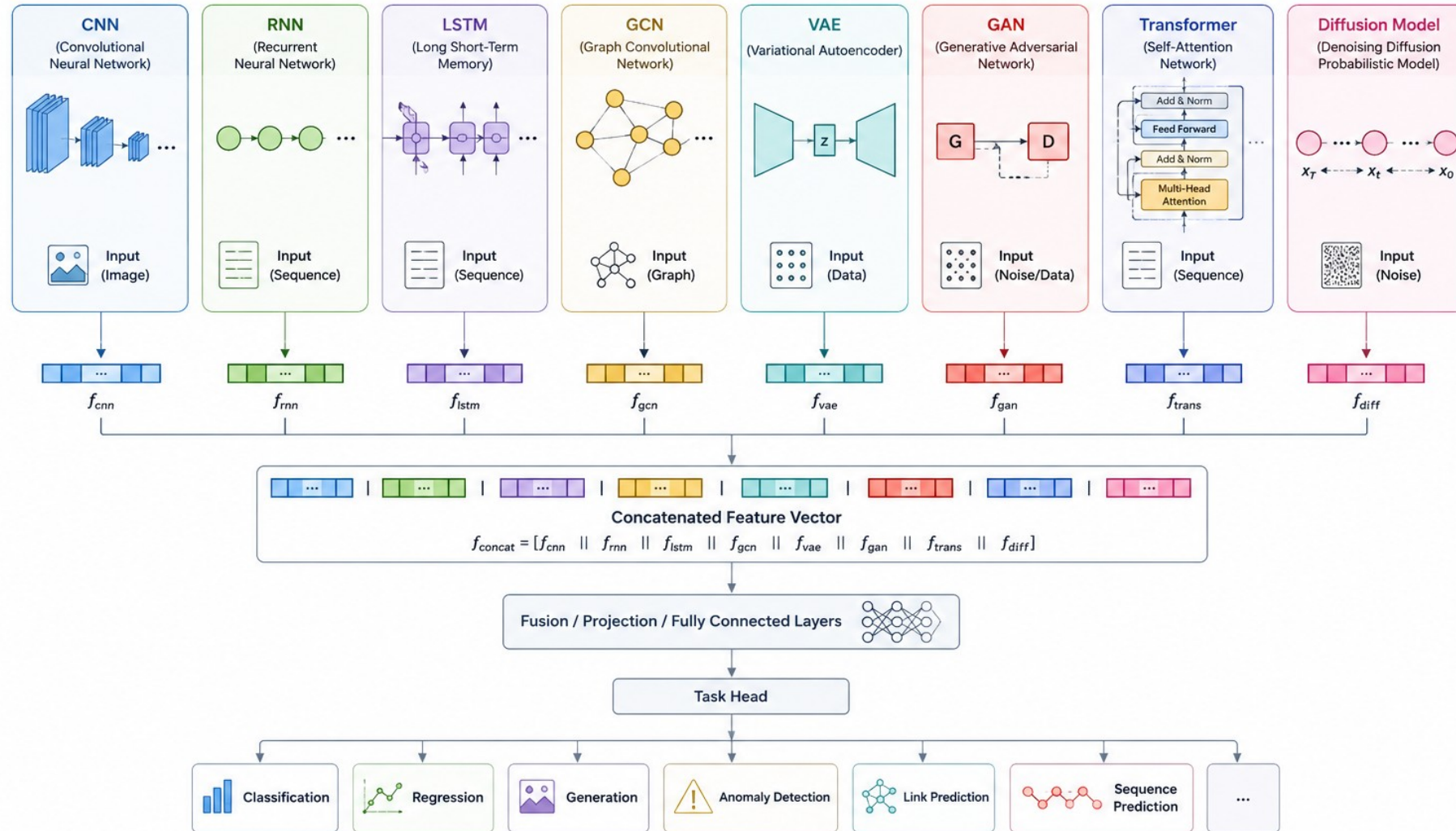
Suppose we made 1000 experiments, 100 were said to be Naval Vessel but actually only 80 of them were Naval, 16 of them were cargo, 4 of them were Passenger ship. For the cargo and the passenger vessels, the numbers are shown above.



Concatenating Models

Concatenating Diverse Models

A Unified Architecture for Multi-Modal Representation Learning



This architecture enables leveraging the complementary strengths of CNN, RNN, LSTM, GCN, VAE, GAN, Transformer, and Diffusion models by concatenating their learned representations for improved performance on complex tasks.



Data Augmentation

UNDERSTANDING DATA AUGMENTATION: TECHNIQUES, MECHANISMS, MATRIX OPERATIONS, & APPLICATIONS

DATA AUGMENTATION STRATEGIES

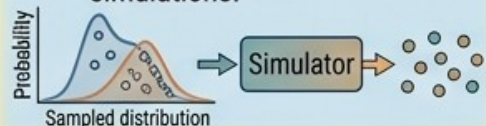
1 FINDING NEW WAYS TO COLLECT DATA

Strategies for novel data acquisition.
→ Replaces 'Early Stopping' plot.



3 GENERATING DATA USING MONTE CARLO SIMULATOR

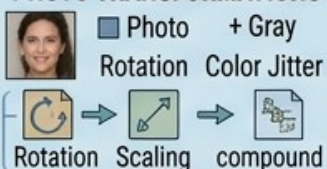
• Generating data through simulations.



4 AUGMENTING USING DATA RELATED TECHNIQUES

• Directly transforming existing data for variety. $L_{xD} = \sum_i w_i$

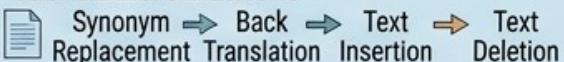
PHOTO TRANSFORMATIONS



SOUND WAVE TRANSFORMATIONS

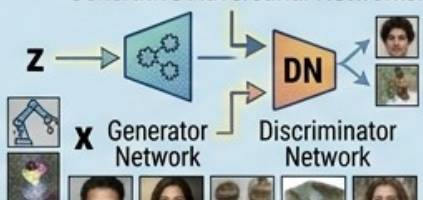


TEXT TRANSFORMATIONS

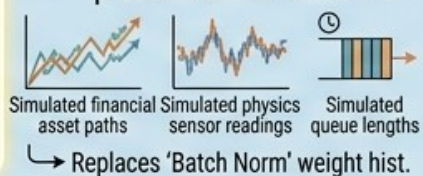


2 GENERATING DATA USING GANs

Creating synthetic data with Generative Adversarial Networks.

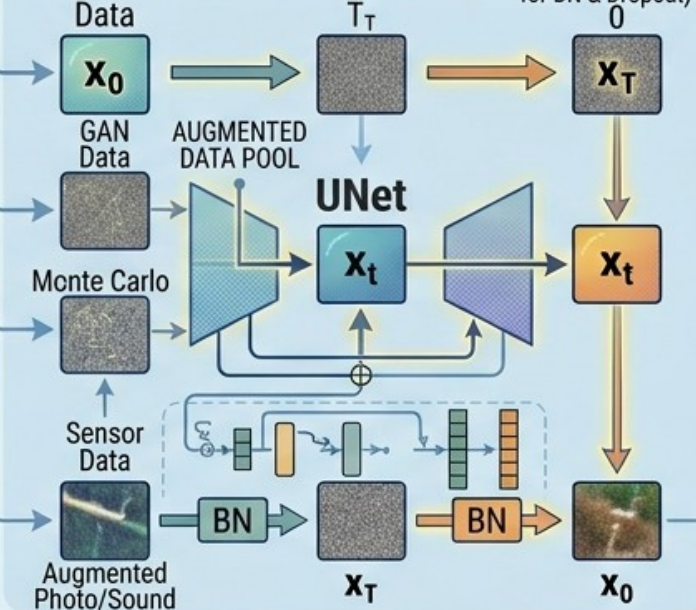


Examples "Dropout" mini-net.
• Generating data through probabilistic simulations.



MECHANISMS & ARCHITECTURE INTEGRATION

AUGMENTED DATA → SIMPLE MODEL → (with added layers for BN & Dropout)



MATRIX OPERATIONS (AUGMENTATION EFFECTS)

1 IMAGE ROTATION EFFECT

$$[x]_{\text{rotated}} = [x] \times [R]^T \text{ where } \begin{pmatrix} 2 & 2 \\ 2 \times 2 \end{pmatrix}$$

$T \times D$ grid → $T \times D$ grid

2 SOUND PITCH SHIFT EFFECT

$$[J] = \begin{pmatrix} T \times D \end{pmatrix} \rightarrow \begin{pmatrix} T \times S \end{pmatrix}$$

$T \times D$ grid → $T \times S$ vector

3 TEXT SYNONYM REPLACEMENT EFFECT

specific word vectors = replaced with neighbor vectors - vocabulary space

$T \times D$ grid → $(1 \times D)$ vector

AUGMENTATION METHOD APPLICATIONS

1 ADVANCED FACE CLASSIFICATION

• From collected, GAN, and transformed



2 ROBUST VOICE RECOGNITION

• From transformed sound files



3 NATURAL LANGUAGE PROCESSING

• From transformed text



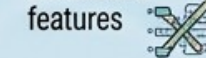
4 COMPUTER VISION TRAINING

• Robustness to new data perspectives



5 DATA DENOISING

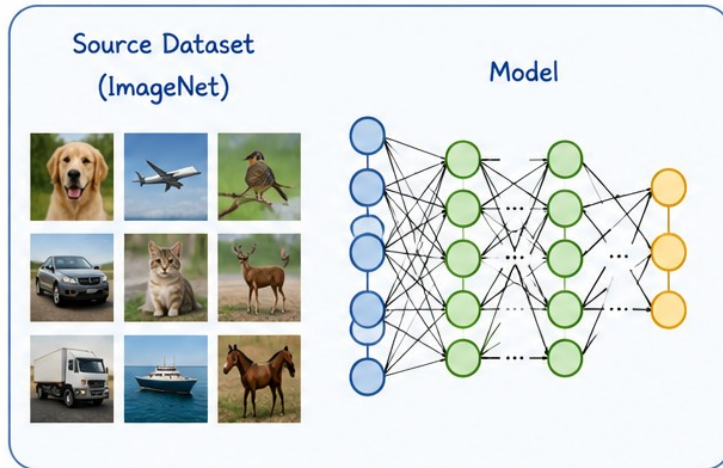
• Batch Norm for consistent features



Transfer Learning

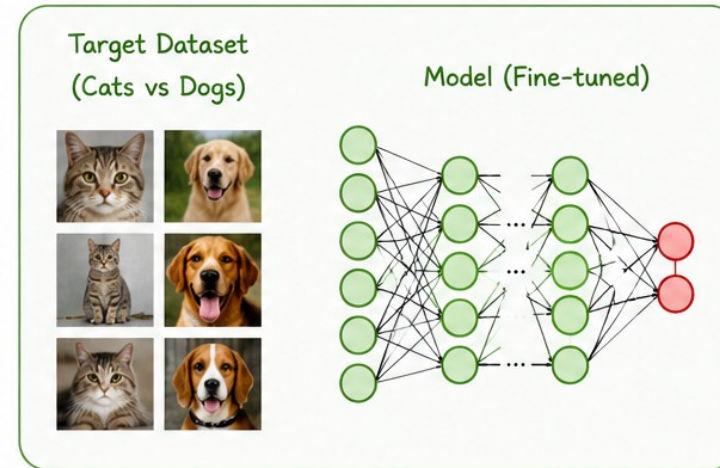
TRANSFER LEARNING

1. Pre-train on Source Task
(Large Dataset)



The model learns **general features**
(edges, textures, shapes, etc.)

2. Fine-tune on Target Task
(Smaller Dataset)



The model **adapts to the target task**
with a smaller dataset

Transfer learned
knowledge
(Keep learned
weights)

Key Idea

Leverage knowledge learned from a large **source task**
and apply it to a related **target task**.



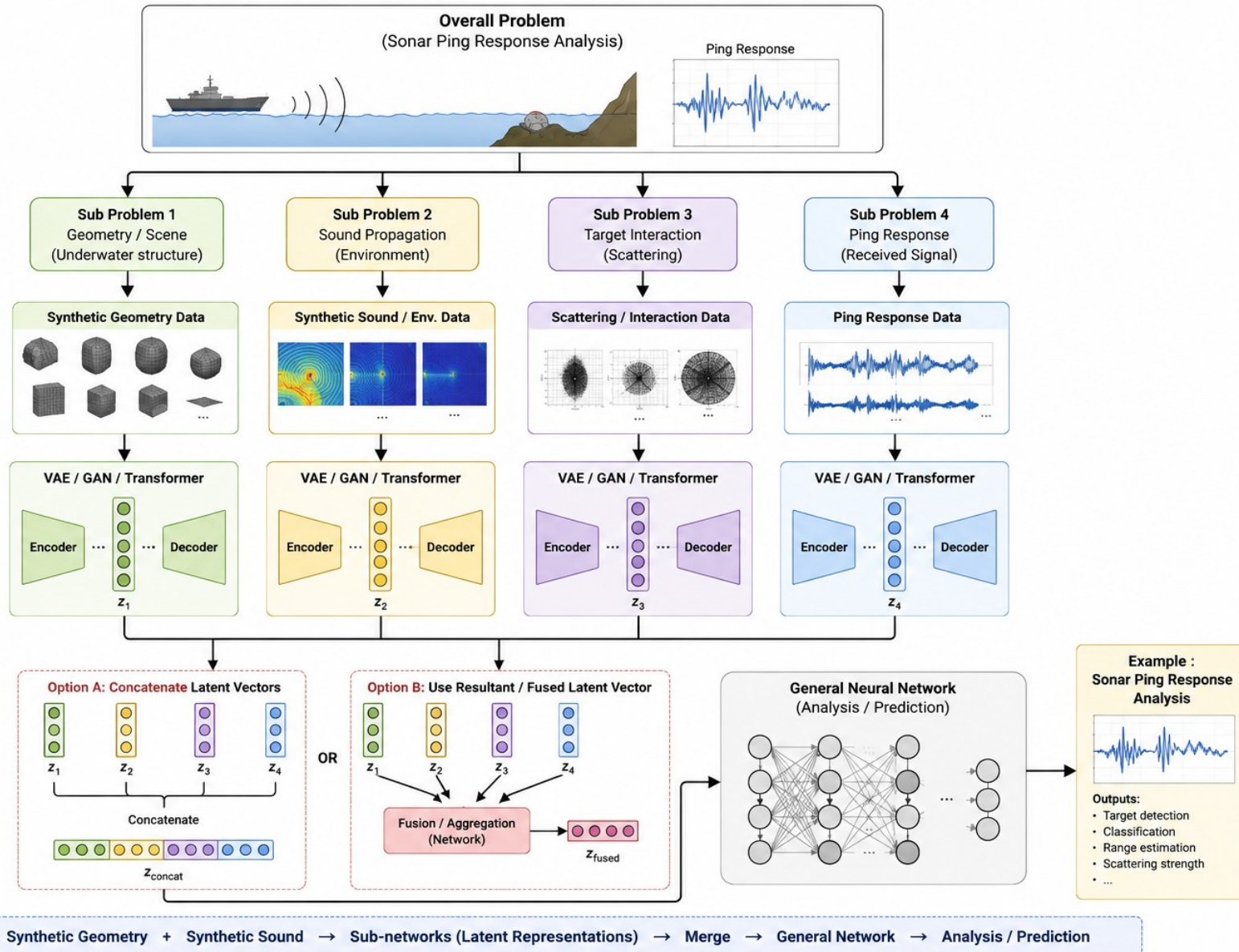
Solving Problems Using Neural Networks

1 Divide the problem into sub problems

2 Prepare generated/real data for sub problems

3 Use a VAE, GAN or Transformer to obtain latent vector for that sub problem

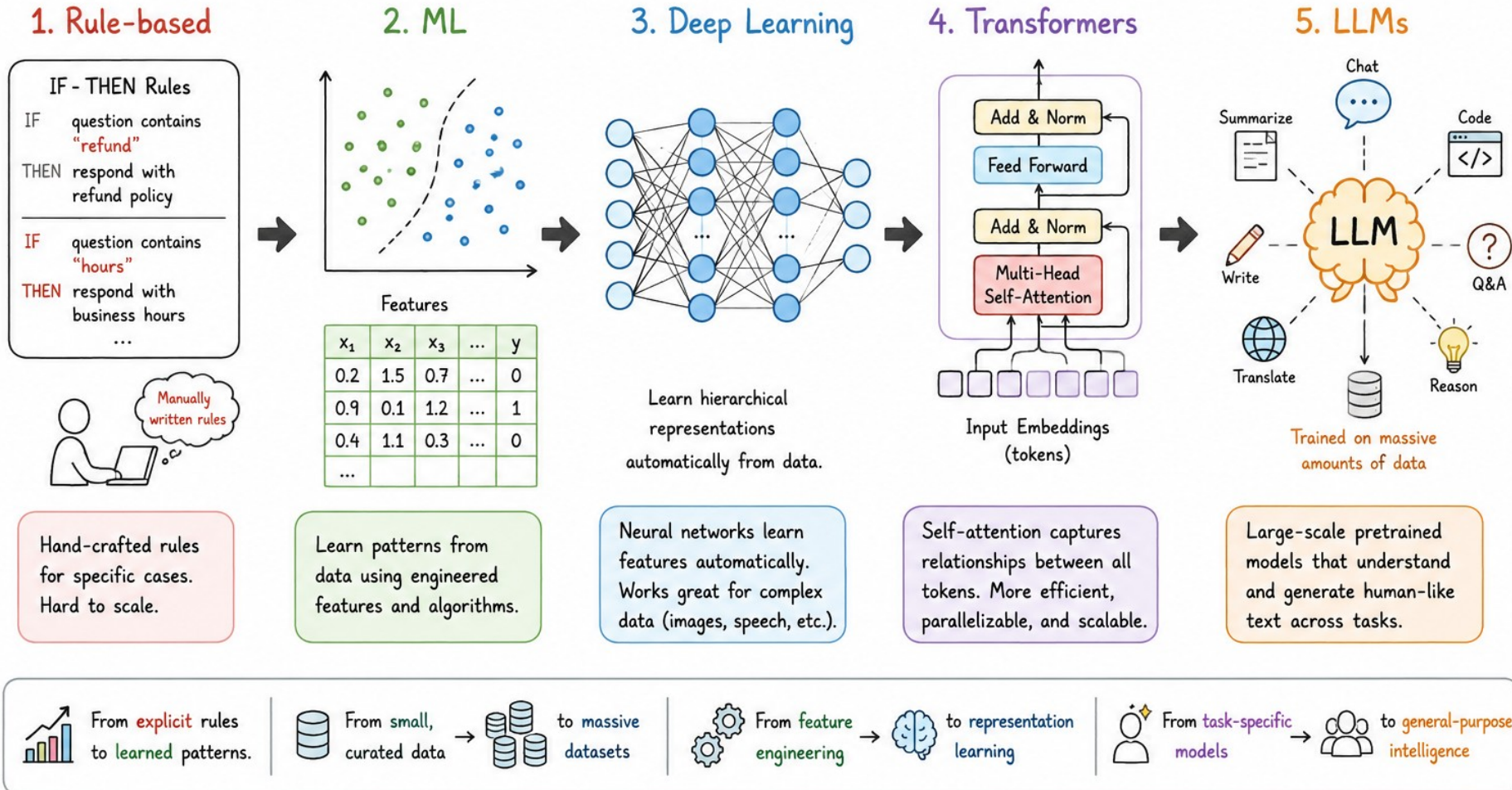
4 Use that network and either concat or use the resultant latent vector for the general neural network



Rule Based → ML → Deep Learning → Transformers → LLMs

What changed ML to LLMs?

Rule-based → ML → Deep Learning → Transformers → LLMs



Transformers

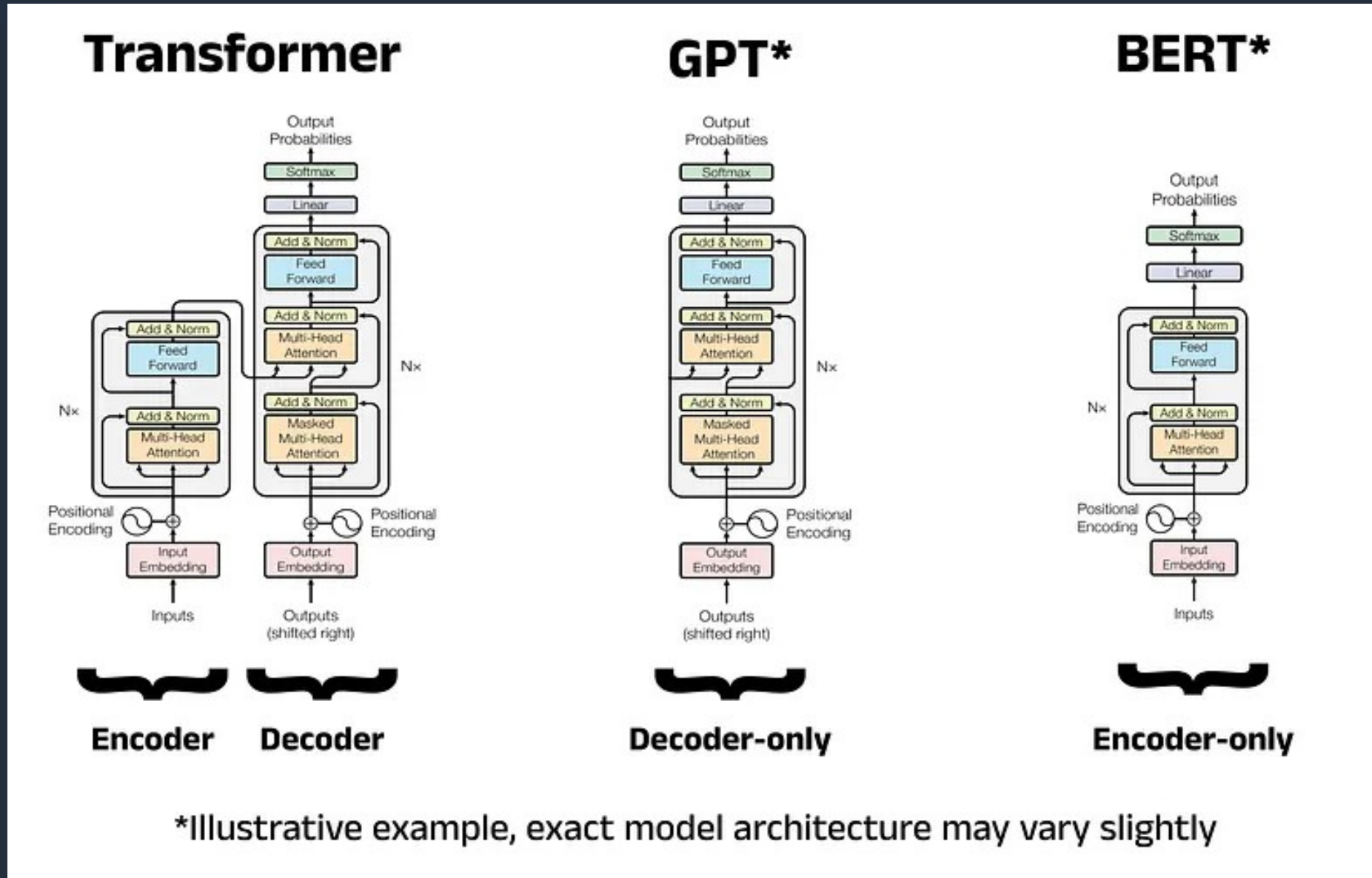
Several massive everyday services use BERT to power their underlying search, recommendation, and text classification systems.

GPT : Gemini, ChatGPT
Billions of parameters

BERT: Google Search, Amazon Search. Max 350 million parameters.

A GPT with 7 billion parameters has roughly:

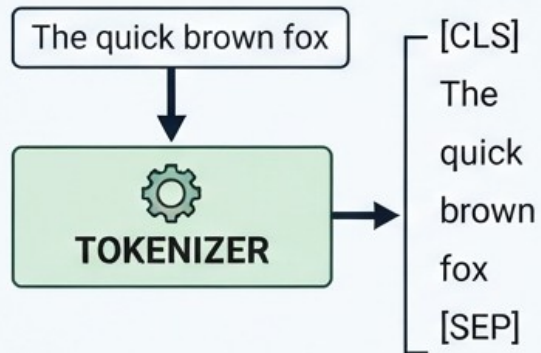
- 32 layers
- hidden size ≈ 4096
- attention heads ≈ 32



Tokenization , Word Embeddings and Positional Embeddings

TRANSFORMER INPUT EMBEDDING PROCESS: TOKENIZATION, WORD & POSITIONAL EMBEDDINGS

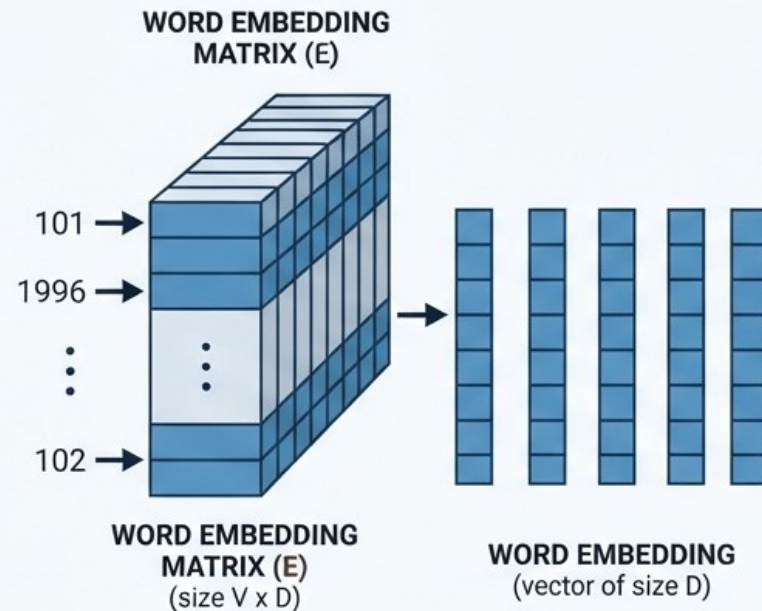
1. TOKENIZATION



VOCABULARY INDEX (LOOKUP TABLE ADDRESSES)

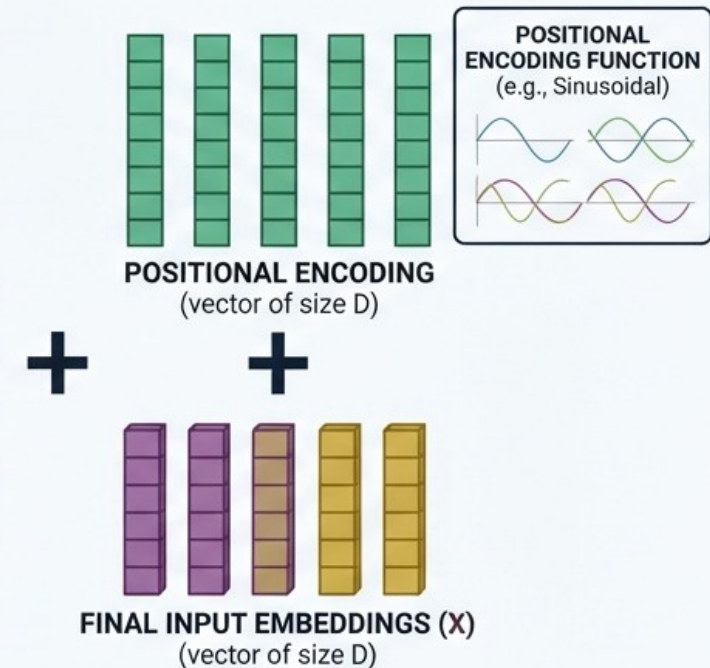
[CLS] = 101
"The" = 1996
"quick" = 4248
"brown" = 2829
"fox" = 4419
[SEP] = 102

2. EMBEDDING LOOKUP



TRAINABLE PARAMETERS

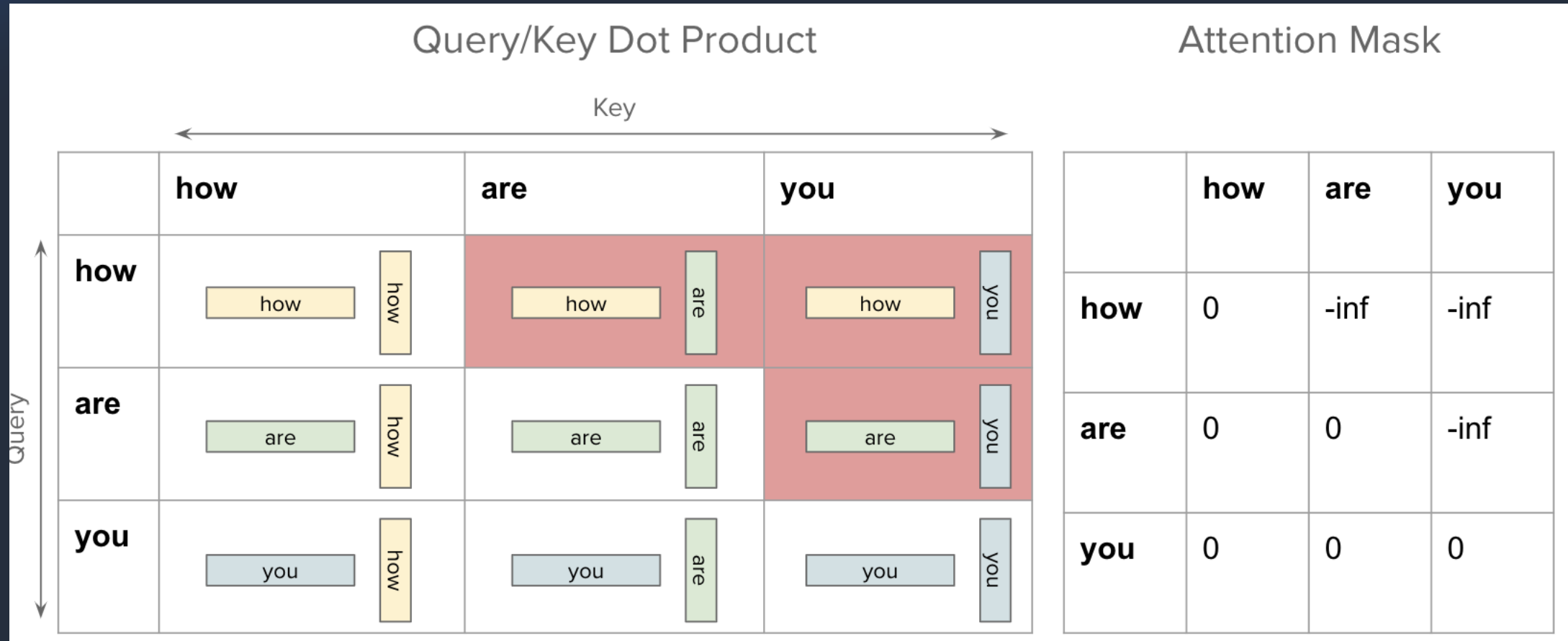
3. POSITIONAL ENCODING ADDITION



TO FIRST
TRANSFORMER BLOCK
(SELF-ATTENTION)

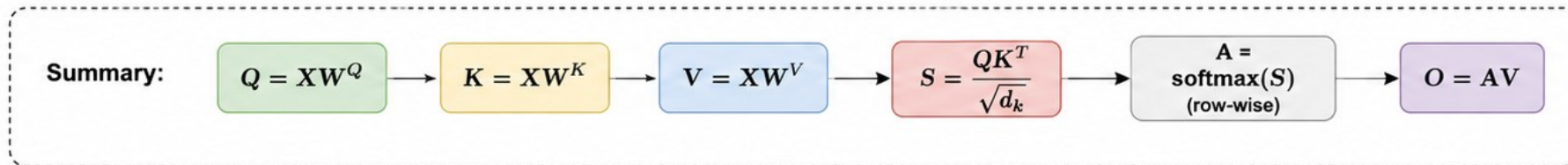
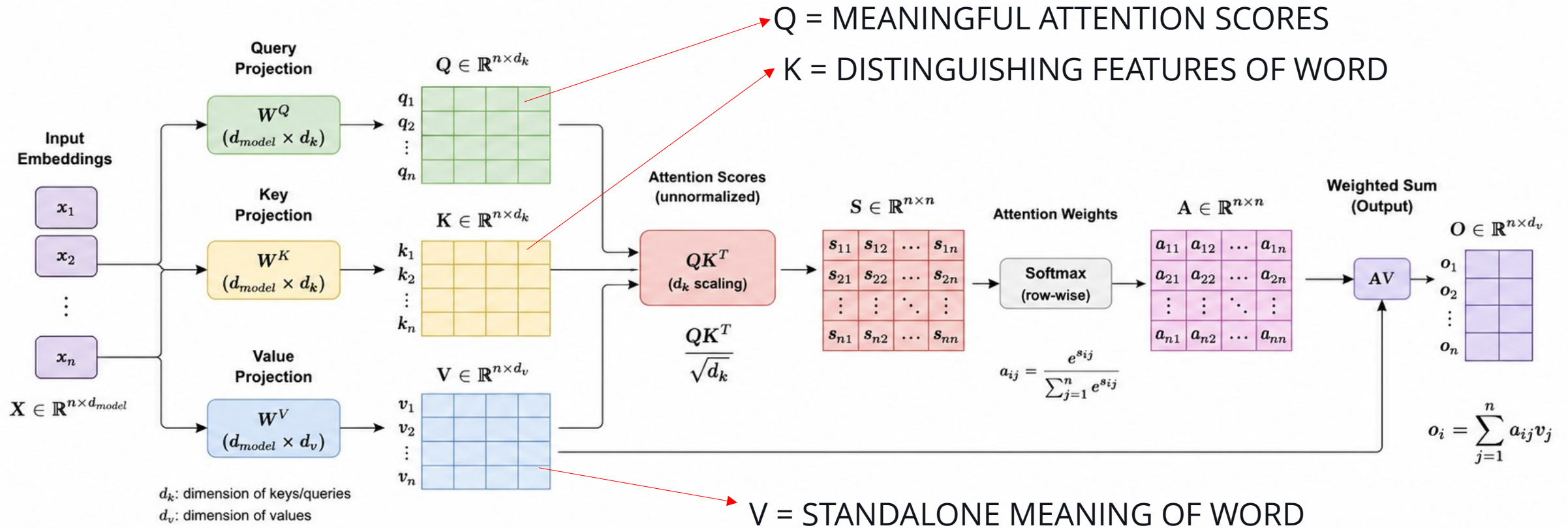


Self-Attention Mechanism



Self-Attention Mechanism

Self-Attention Mechanism



Output O captures context-aware representations of the input sequence.



Context Window

128KB context takes **20GB** memory in Qwen 3.6 35B

Context Window Size is directly related to the KQV matrices.

When we augment a context size, the VRAM consumption will also increase quadratically (L^2)

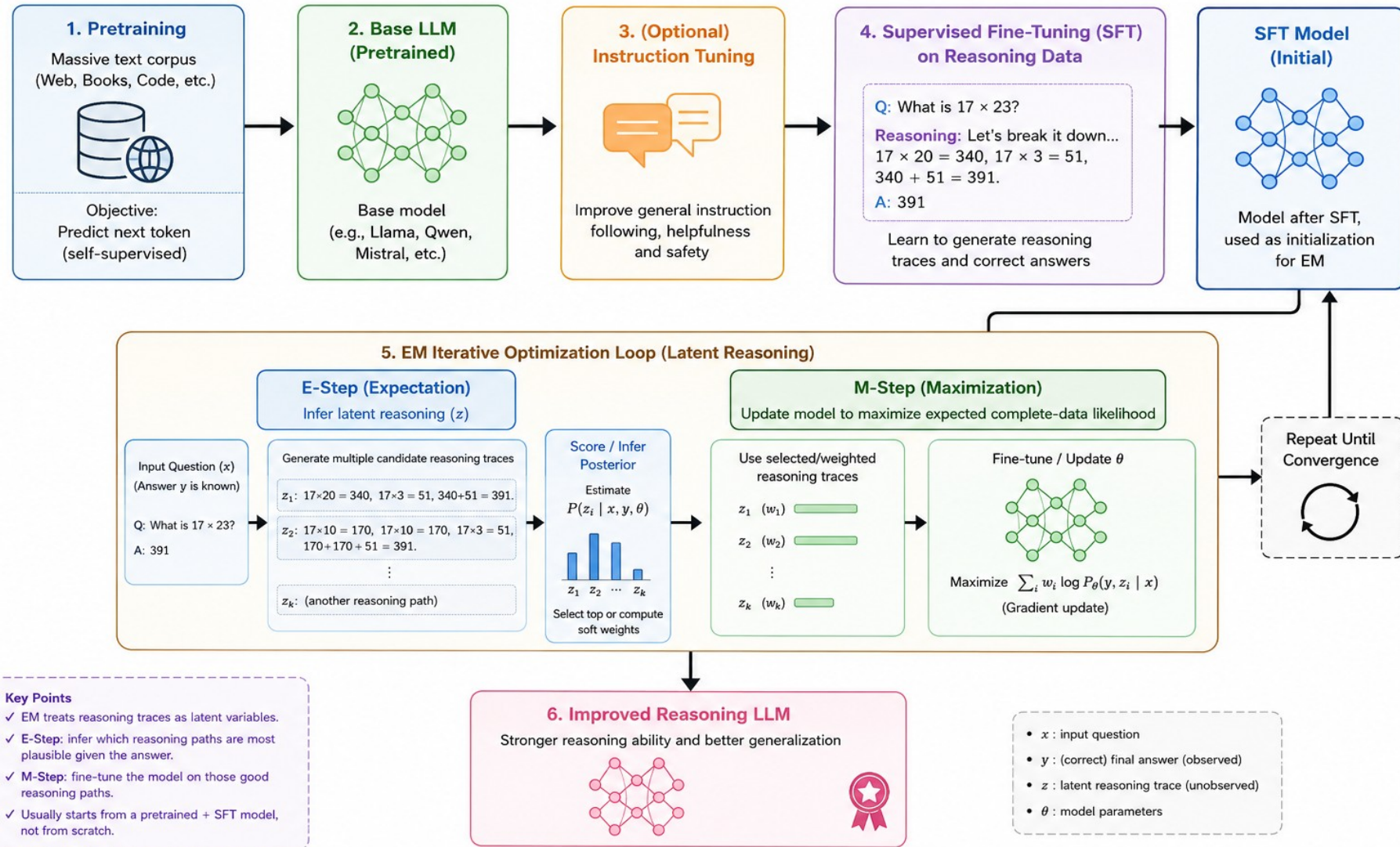
GOOGLE-TITANS : RNN/Transformer Mix. 2 Million Token Context Size



REASONING

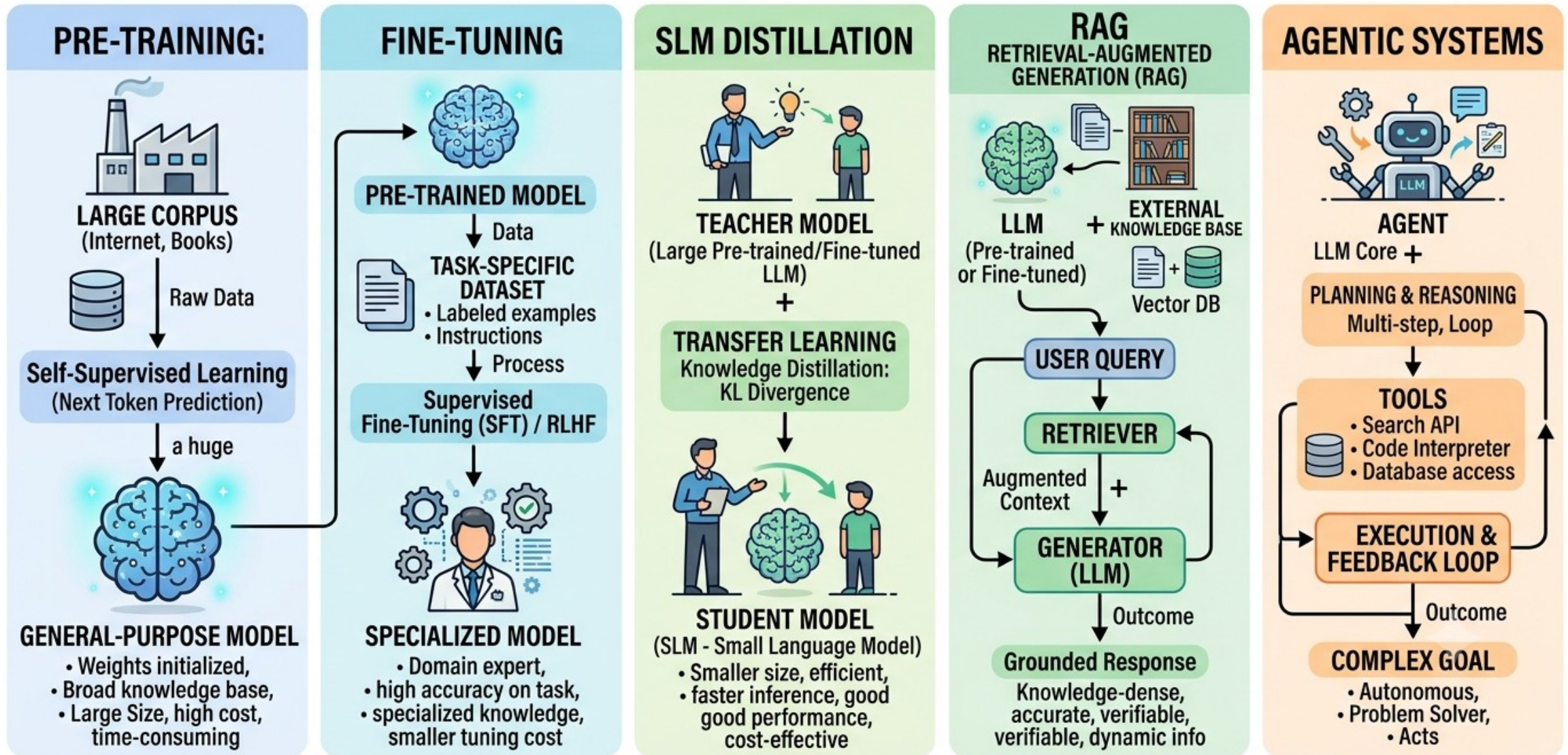
OPEN AI 2025 : Learning to Reason in LLMs by Expectation Maximization

LLM Training Pipeline with EM for Reasoning



Pre-Training vs Fine Tuning vs SLM Distillation vs RAG vs Agentic

COMPARATIVE OVERVIEW: PRE-TRAINING vs. FINE-TUNING vs. SLM DISTILLATION vs. RAG vs. AGENTIC SYSTEMS



Pre-Training vs Fine Tuning vs SLM Distillation vs RAG vs Agentic

Approach	Compute & Hardware Cost	Development & Training Time	Required Data Size / Source	Primary Purpose
Pretraining	Very High (\$100k - \$10M+)	Months / Years	Trillions of tokens (Terabytes)	Teaching a model language and world knowledge from scratch
SFT	Low / Medium (\$100 - \$10k)	Days / Weeks	1k - 100k high-quality samples (Megabytes)	Teaching a model format, style, and specific tasks
RAG	Low (Cloud & Vector DB costs only)	Days	Unlimited enterprise documents (Gigabytes)	Providing a model with dynamic, up-to-date, and private data
Agentic	Very Low / Medium (Software development cost)	Weeks / Months	No training data needed (API/Tool docs)	Turning a model into an autonomous, reasoning system



Inference vs Training Cost

AI COST COMPARISON: TRAINING vs. INFERENCE (8x L40S GPUs | 2 Servers)

TRAINING PHASE



COMPUTE COSTS (GPUs/HPC)
HIGH (\$10M+ to \$100M+)
Thousands of H100s for months

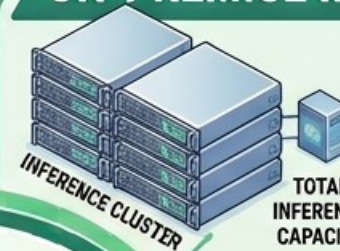
DATA ACQUISITION & CURATION
SIGNIFICANT (\$1M - \$10M+)
Petabytes of diverse data

ENGINEERING & TALENT
HIGH (\$Millions)
Expert teams for months/years

ESTIMATED TOTAL TRAINING COST: \$50 Million

*Illustrative example for a large language model.

ON-PREMISE INFERENCE DEPLOYMENT: MOCDB



MODEL: QWEN 3.6 35B | CONTEXT: 128k

VRAM ALLOCATION & CAPACITY (Qwen 35B / 128k Context)
Example Model: Qwen 3.6 35B Quantized (e.g., ~16GB VRAM)
Model & Overhead per GPU (L40S 48GB): e.g., ~32GB (for 128k context support)
Active User Capacity per GPU e.g., ~1.8 (using batching)
Total Simultaneous Users (Cluster): 12 Users (8 GPUs * 1.5 per GPU)
Total Cluster VRAM (8x L40S GPUs) **384GB**

RECOMMENDED HARDWARE (Per Server Hardware (5x))
GPUs: 8x NVIDIA L40S (2 servers each with 4 GPUs, 96 Tensor Cores per GPU)
CPUs: 2x Data Center CPUs (Per Server) (High Core Count, e.g., 64-core AMD EPYC)
RAM: 512GB DDR5 (High Bandwidth) per server
Networking: Dual-port 100GbE / InfiniBand per server

ANNUAL DEPRECIATION & HARDWARE REFRESH
\$MODERATE (e.g., \$60k / year)

POWER, COOLING & FACILITY COSTS
\$MODERATE (e.g., \$30k / year)

ENGINEERING & IT STAFF (Per Server basis)
\$MODERATE OPEX: e.g., \$150k / year

STORAGE & BACKUP
\$Low (e.g., \$5k / year)

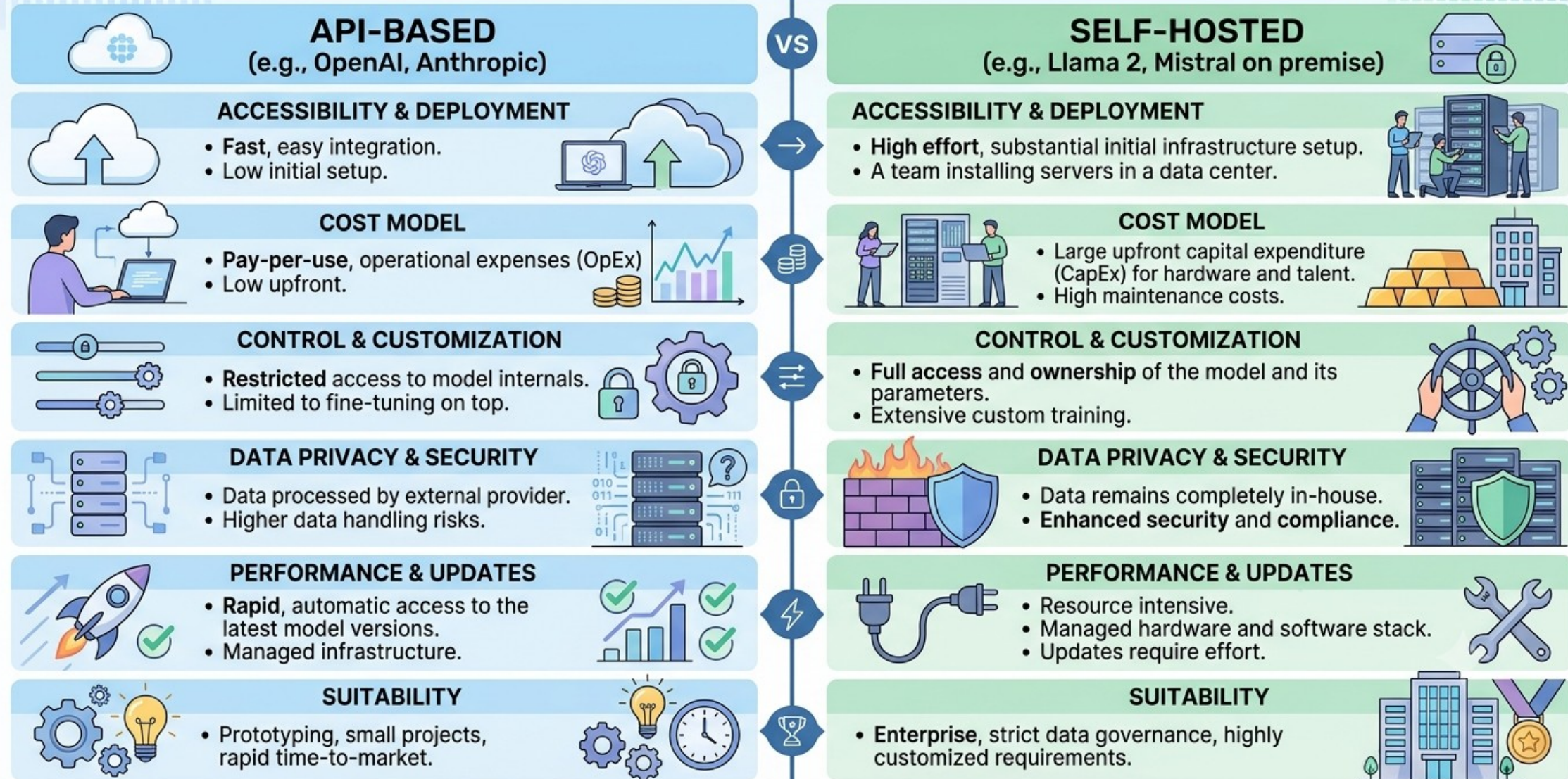
ESTIMATED ANNUAL CLUSTER OPEX: e.g., \$245k (including staffing)

*Detailed on-premise infrastructure calculation and costs based on typical data center operations and hardware lifecycle.



API VS SELF-HOSTED

API-BASED vs SELF-HOSTED LLM COMPARISON



API VS SELF-HOSTED

- API Usage :
 - The following providers are available :
 - OPEN AI (GPT)
 - GOOGLE (GEMINI)
 - ANTHROPIC (CLAUDE)
 - xAI (GROK)
 - DeepSeek (DeepSeek)
 - Z.AI (Zhipu)
 - Alibaba (QWEN)
 - Mistral AI (Mistral)
 - Cohere (Cohere)
 - Get an API Token Key and use it in the client software installed on the developers machine.
 - General token consumption ranges from \$100 to \$2000 per month for a developer.



Prompt Structure

GOAL : Write precise prompts to prevent hallucinations

System Prompt (Prompt Engineering)

Role/Persona : APC/SPC/WCC, Coder/Analyst ...

Mode : Plan Mode, Execute Mode, ...

Constraints :

Rules

Boundaries

Context/Reference Data

RAG

Ontology

LLM Wiki

User Query

Chain of Thought (COT)

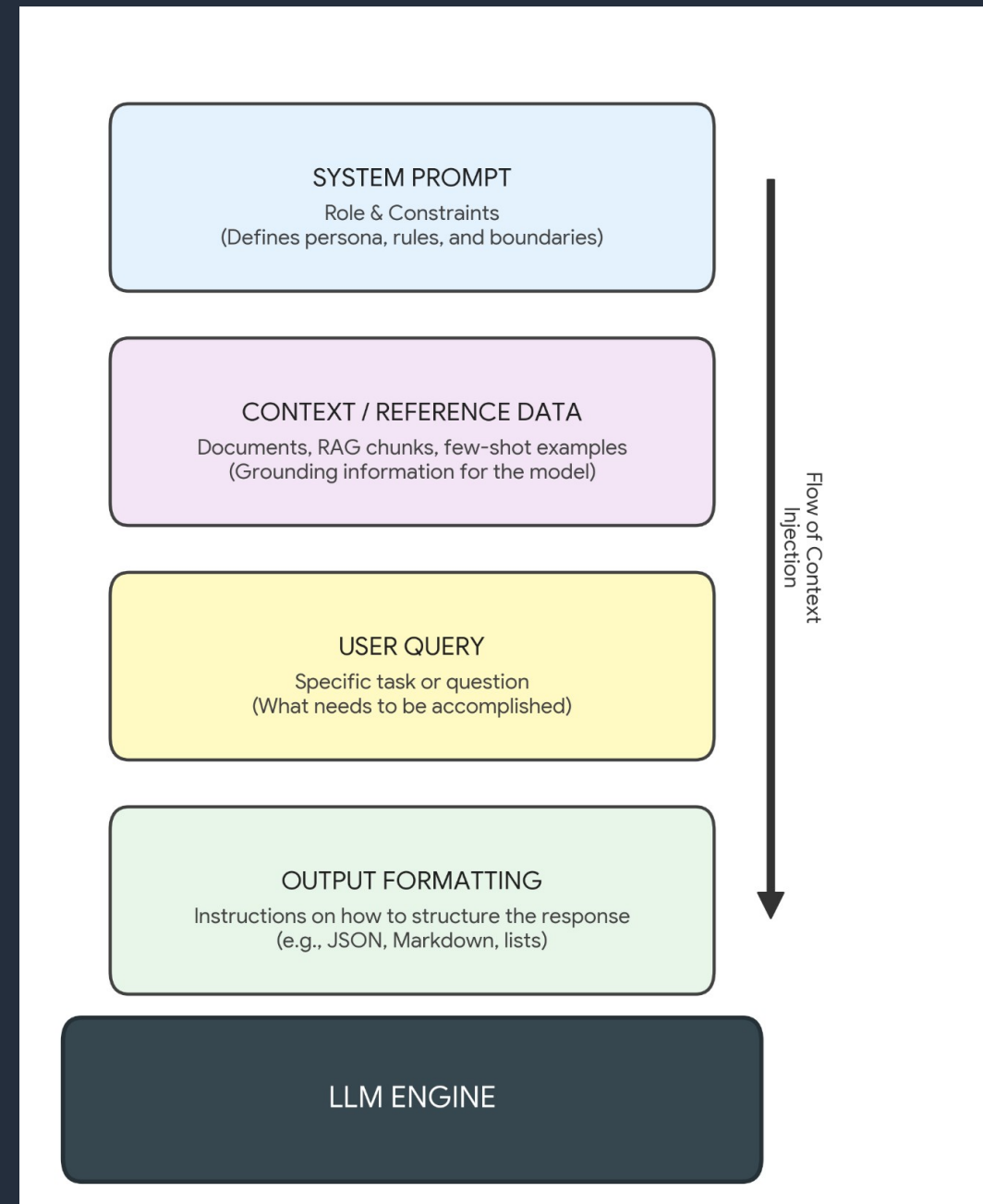
Input Data (Previous mails, examples) IN PARANTHESIS

Explicit Negative

Venn Diagram of what is possible

Intersection, Union, Exclusion

Ouput Formatting Directives : JSON, MD, HTML, ...

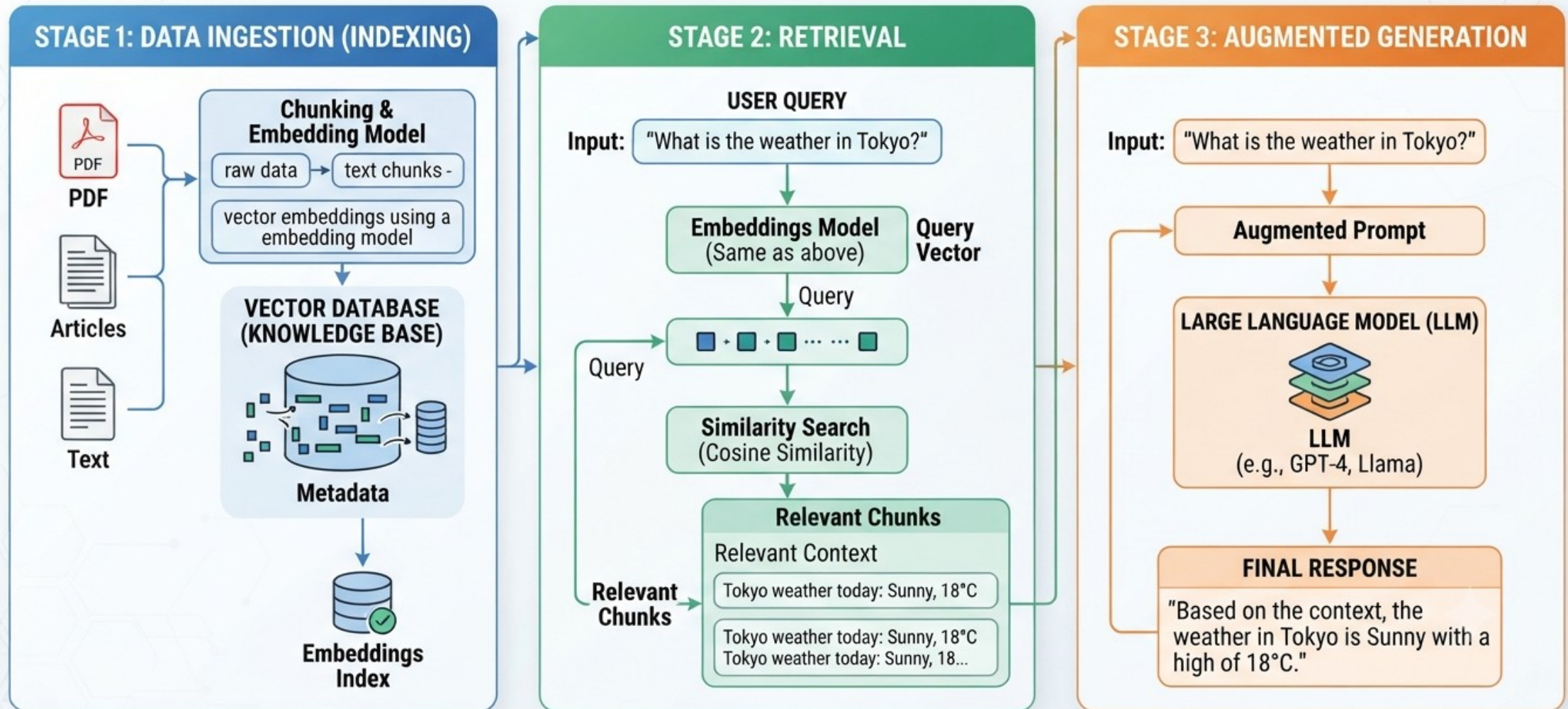


Prompt Anti Patterns

1. Don't spam prompts to fix errors (Prompt Thrashing).
2. Don't assume model is understanding (Write Clear Prompts)
3. Don't use too many MCP Servers (Risk of selecting wrong MCP would increase)
4. Regularly reevaluate (possibly outdated) assumptions about the "best" tools
5. Don't persist with dead-end conversations



RETRIEVAL-AUGMENTED GENERATION (RAG) MECHANISM



UNDERSTANDING RAG: BENEFITS, RISKS, & LIMITATIONS

WHAT YOU GAIN - BENEFITS OF RAG



1. **ACCURACY & FACTUALITY:** Grounded in data. Reduces Hallucinations.



2. **UP-TO-DATE KNOWLEDGE:** Access to latest information without re-training.



3. **SOURCE CITATION:** Links answers back to source documents.



4. **DOMAIN-SPECIFIC EXPERTISE:** Handles specialized vocabularies effectively.



5. **COST-EFFICIENT:** Avoids costly LLM re-training.

WHAT YOU HAVE TO BE CAREFUL ABOUT - RISKS & CHALLENGES



1. **DATA PRIVACY:** Ensuring sensitive data is protected in the Vector DB.



2. **SOURCE BIAS:** Relying on biased or outdated source documents.



3. **SYSTEM COMPLEXITY:** Integrating multiple components (retriever, generator).



4. **LATENCY (SPEED):** Multiple steps increase response time.



5. **HALLUCINATION RISKS:** Still possible, though reduced, especially with poor retrieval.

WHAT ARE THE PITFALLS - IMPLEMENTATION & OPERATIONAL ISSUES



1. **POOR RETRIEVAL QUALITY:** Retrieving irrelevant or incomplete context.



2. **INCORRECT CHUNKING:** Optimal chunk size and overlap is critical.



3. **INACCURATE EMBEDDINGS:** Poor embedding model choices.



4. **DATA STALENESS:** Failure to keep the Vector DB updated.



5. **SCALABILITY:** Managing a massive and growing knowledge base.

WHAT ARE THE SHORTCOMINGS - INHERENT LIMITATIONS



1. **RELIANCE ON RETRIEVED DATA:** Can't answer what's not in the DB.



2. **LLM GENERATION ERRORS:** LLM might misinterpret perfect context.



3. **LIMITED REASONING CHAIN:** Difficulties with multi-hop reasoning.



4. **CONTEXT WINDOW LIMIT:** Hard cap on how much data is fed to the LLM.

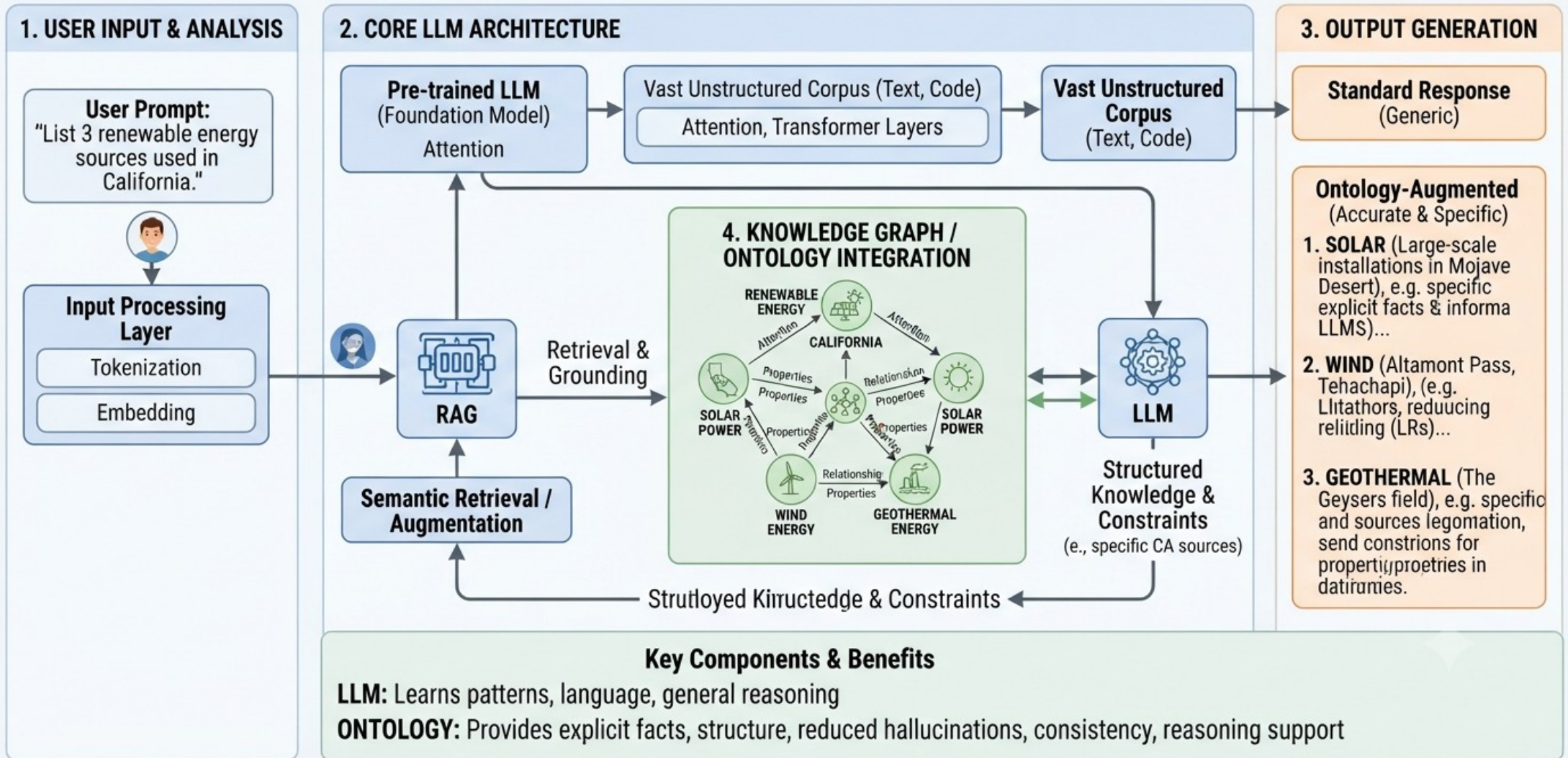


5. **HIGH INFRASTRUCTURE COST:** Maintaining vector DB and specialized hardware.

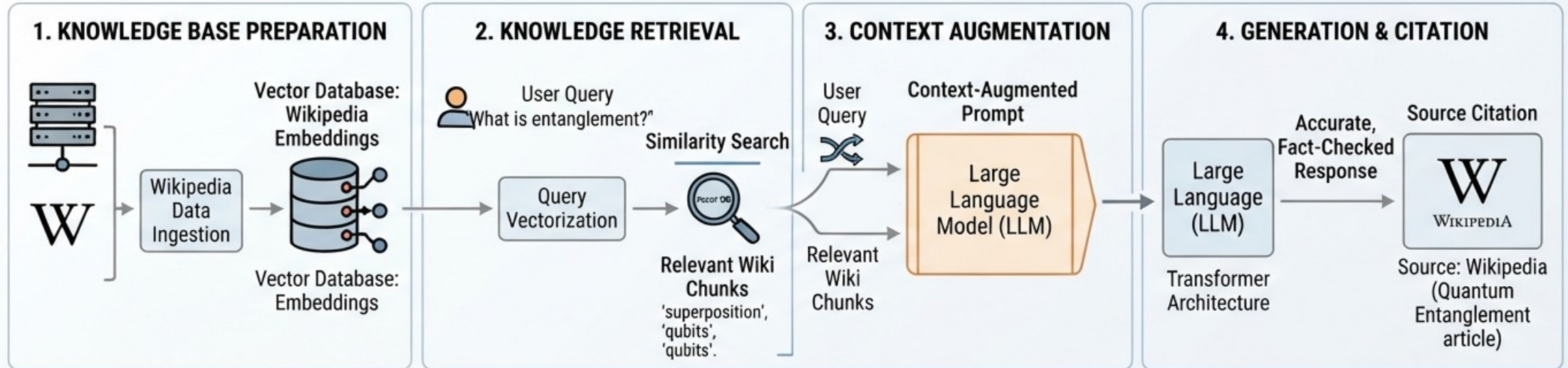
RAG CONTEXT + USER REQUEST may cause context size problems,
this is handles by “Text Chunking” and “Top-K Filtering” !



ONTOLOGY MECHANISM FOR LLMs: ENHANCING KNOWLEDGE & REASONING



LLM WIKI MECHANISM: Knowledge Retrieval & Generation



KEY PROBLEM SOLVED: HALLUCINATION REDUCTION

WITHOUT MECHANISM (Hallucination)



Internal
Memory

Entanglement allows
faster-than-light
communication

Source: General Training Data
(Potential Hallucination)

WITH MECHANISM (Fact-Checking)



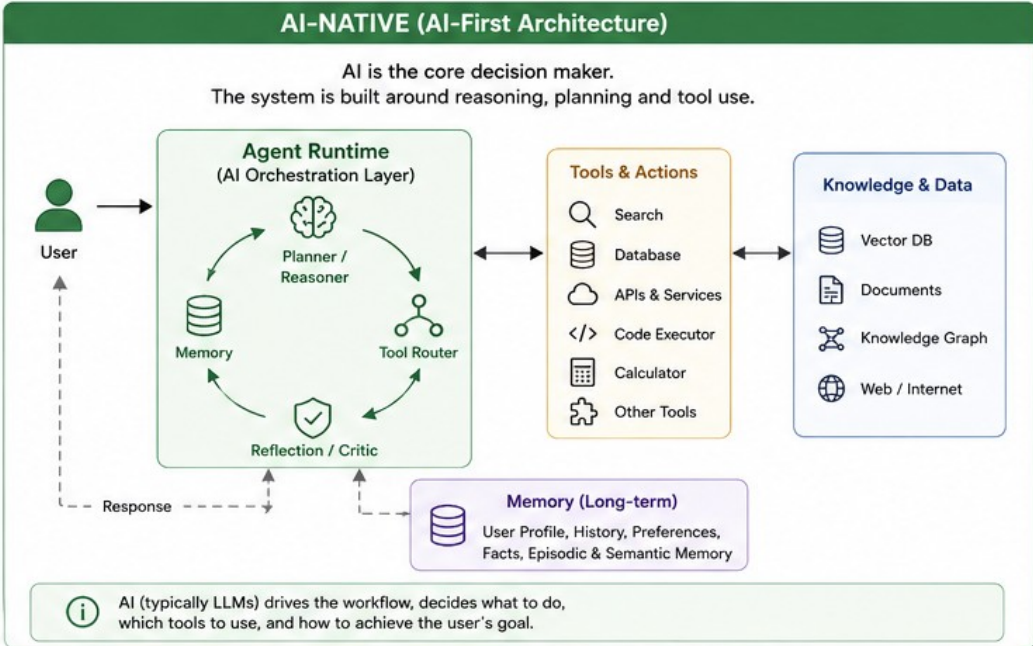
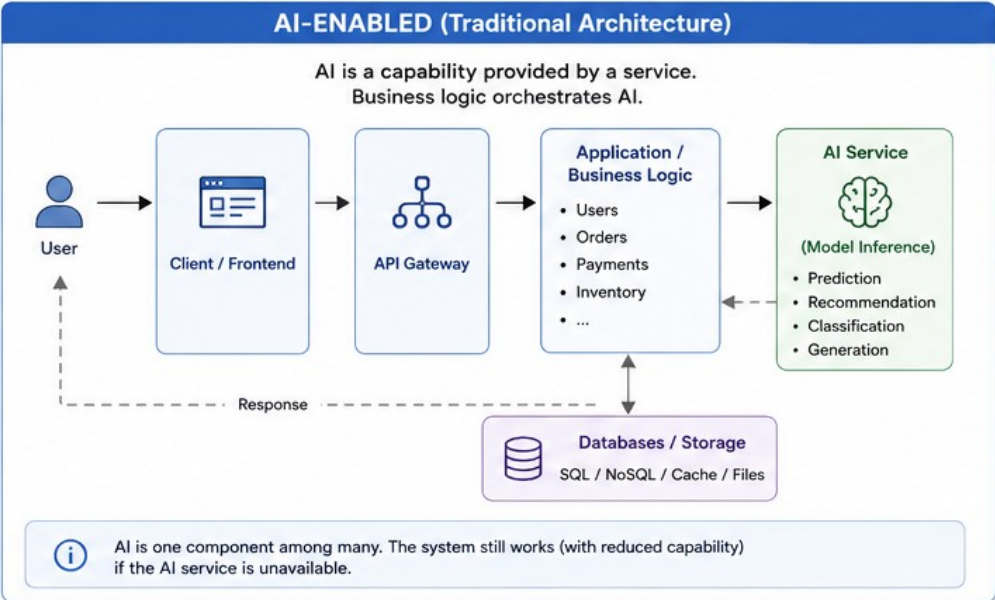
LLM +
Wiki

Entanglement describes
particles correlated regard
of distance, not FTL

Source: Wiki Knowledge Base
(Verified)

AI-ENABLED vs AI-NATIVE SYSTEMS

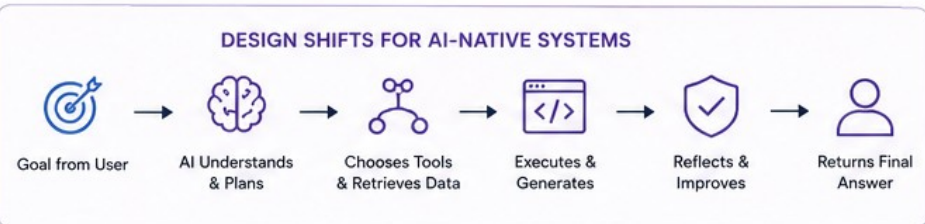
Comparison of architectures, characteristics, and design considerations



KEY COMPARISONS		
Aspect	AI-ENABLED (Traditional)	AI-NATIVE (AI-First)
System Objective	Automate specific tasks / Add intelligence to features	Achieve goals through reasoning and adaptation
Decision Maker	Deterministic business logic	AI (LLM / Agent)
Control Flow	Predefined workflows & rules	Dynamic, goal-driven, AI-planned
AI Role	One of many services (plug-in)	Core orchestrator & decision engine
Data Usage	Primarily transactional data	Contextual, unstructured, knowledge-rich, multi-source
Extensibility	Add new features via engineering	Add new capabilities via tools & prompts
Adaptability	Low - requires code changes	High - adapts through learning, memory & feedback
Evaluation	Accuracy, latency, uptime, business KPIs	Task success rate, goal completion, user satisfaction, safety
Cost Structure	Lower variable cost (predictable)	Higher variable cost (token usage, model calls)
Failure Handling	Fallback rules / Defaults	Reflection, retry, re-plan, human-in-the-loop

ARCHITECTURAL COMPONENTS COMPARISON	
AI-ENABLED COMPONENTS	AI-NATIVE COMPONENTS
Frontend / Client	User Interface (Chat / Voice / etc.)
API Gateway	Agent Runtime / Orchestrator
Application / Business Logic	Planner / Reasoner (LLM)
Databases / Storage	Memory System (Short & Long-term)
AI / ML Service (Inference)	Tool Router
Cache	Tools & Connectors
Monitoring & Logging	Knowledge Base / RAG Layer
Auth / Authorization	Reflection / Critic
Background Jobs	Guardrails & Safety
	Observability & Evaluation

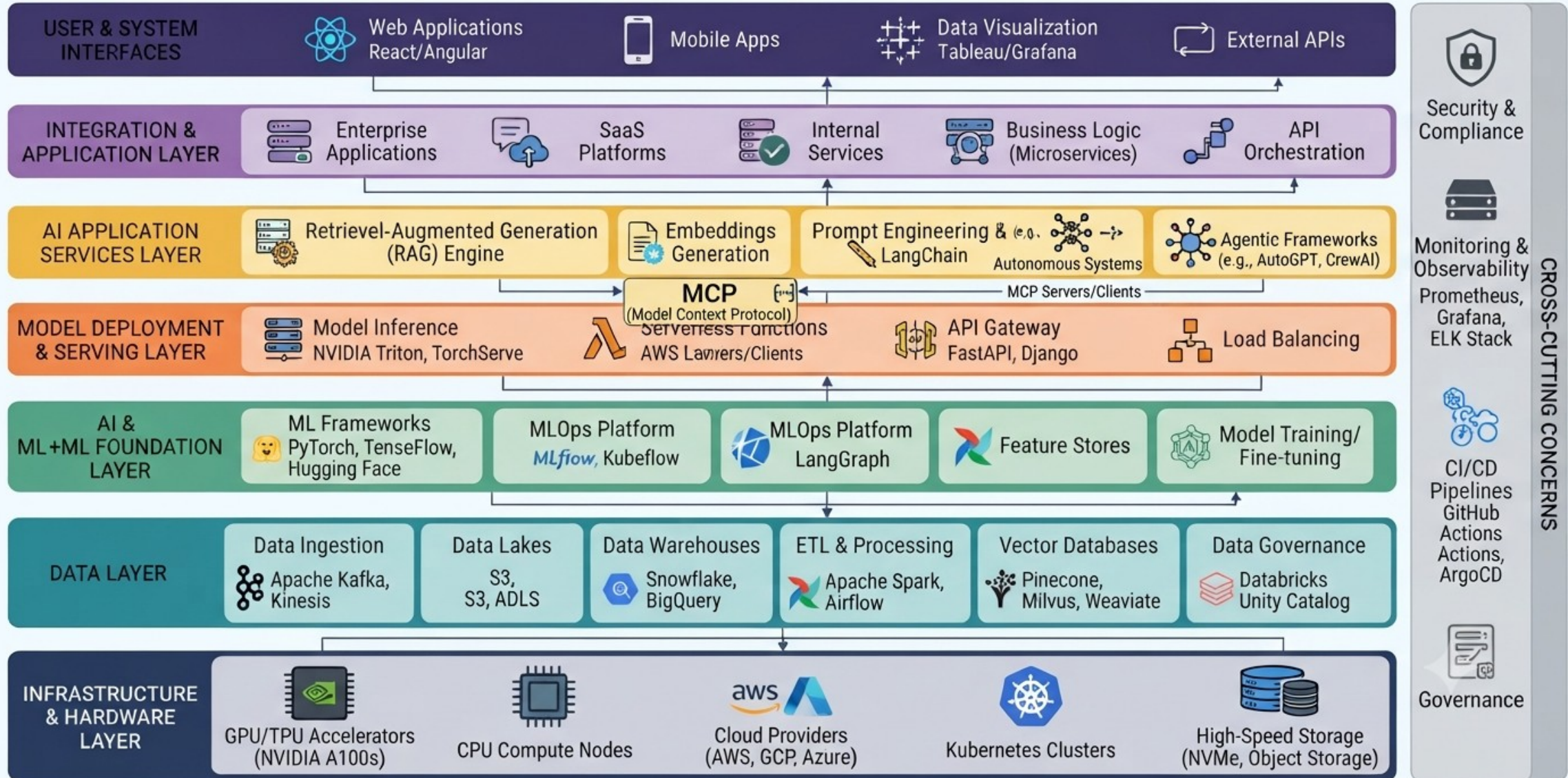
USE CASE EXAMPLES	
AI-ENABLED	AI-NATIVE
• Recommendation in e-commerce • Fraud detection in payments • Smart compose in email • Demand forecasting • Image classification in an app	• AI Personal Assistant • Autonomous Research Agent • AI Software Engineer (Copilot-like) • Customer Support Agent • Multi-step Task Automation



★ KEY TAKEAWAY
AI-enabled systems use AI to improve parts of the product.
AI-native systems are built around AI as the core engine of reasoning, planning and action.

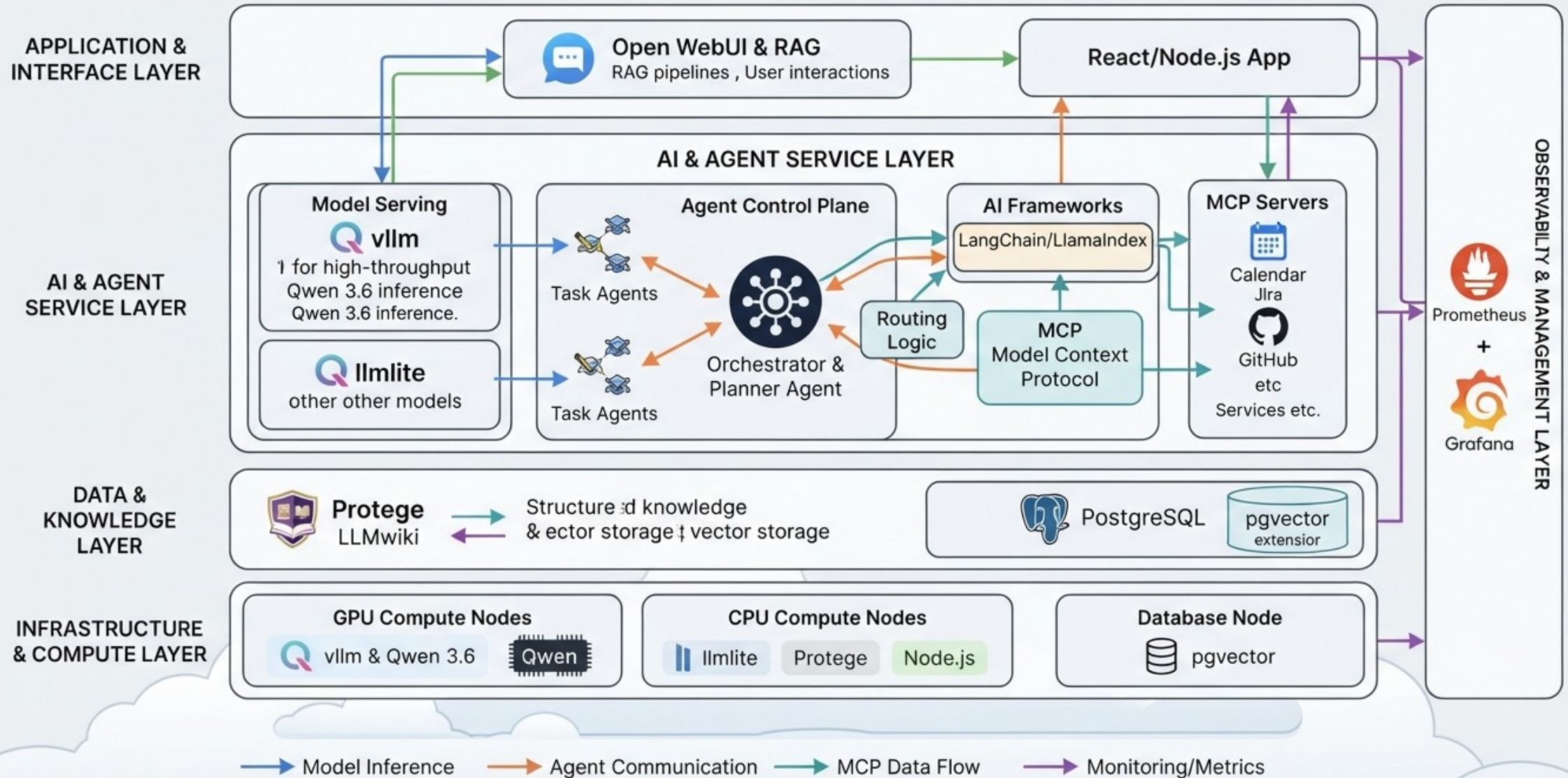
Technology Stack

AI TECHNOLOGY STACK WITH MCP AND AGENTIC FRAMEWORKS IN REAL SOFTWARE ARCHITECTURE DIAGRAM



Where AI Sits In System Design : Example

CONCRETE ARCHITECTURE: AGENTIC AI & RAG PLATFORM STACK



Where AI Sits In System Design : Design Considerations

AI SYSTEM DESIGN CONSIDERATIONS

Key factors to consider when designing AI-enabled or AI-native systems

1. FUNCTIONALITY & GOALS



Define what the system should achieve and how AI contributes to it.

Consider:

- What problems are we solving?
- What decisions should AI make vs. rule-based logic?
- Success metrics & KPIs
- Human-in-the-loop or fully autonomous?

2. DATA



Data is the foundation of any AI system.

Consider:

- Data sources & availability
- Data quality, completeness, consistency
- Bias & fairness in data
- Data labeling / annotation needs
- Data privacy & ownership
- Data storage, retention & lifecycle

3. MODEL & AI APPROACH



Choose the right model(s) and AI approach.

Consider:

- Model type (LLM, ML, CV, RL, etc.)
- Build vs. buy vs. fine-tune
- Model accuracy vs. latency vs. cost trade-offs
- Context length & reasoning needs
- Evaluation strategy
- Model versioning & lifecycle

4. ARCHITECTURE & INTEGRATION



Design how AI components fit into the overall system architecture.

Consider:

- AI-native vs. AI-enabled approach
- Microservices, modularity & APIs
- Orchestration (agents, workflows, tools)
- Integration with existing systems
- Scalability & elasticity
- Multi-model routing & fallback

5. PERFORMANCE & COST



Balance performance requirements with operational cost.

Consider:

- Latency requirements (p95, p99)
- Throughput & concurrency
- Infrastructure (GPU/CPU) needs
- Token usage & model cost
- Caching, batching, and optimization
- Cost monitoring & budgeting

6. RELIABILITY & RESILIENCE



Ensure the system is reliable, available and can handle failures gracefully.

Consider:

- High availability & fault tolerance
- Timeouts, retries & fallbacks
- Graceful degradation strategies
- Model / service redundancy
- Disaster recovery & backup
- Testing & chaos engineering

7. SECURITY & PRIVACY



Protect data, models and users throughout the system.

Consider:

- Authentication & authorization
- Data encryption (in transit & at rest)
- Prompt injection & jailbreak protection
- Secrets & key management
- PII detection & data masking
- Compliance (GDPR, HIPAA, SOC2, etc.)

8. SAFETY, FAIRNESS & ETHICS



Build responsible AI that is safe, fair and aligned with human values.

Consider:

- Bias, fairness & inclusivity
- Toxicity, harmful content mitigation
- Content filtering & guardrails
- Human oversight & escalation
- Transparency & explainability
- Alignment with policies & values

9. OBSERVABILITY & EVALUATION



Continuously monitor, evaluate and improve system and model performance.

Consider:

- Logging, tracing & metrics
- Model performance monitoring
- Data & concept drift detection
- User feedback & quality evaluation
- A/B testing & experimentation
- Alerting & anomaly detection

10. HUMAN EXPERIENCE



Design for trust, usability and effective human-AI collaboration.

Consider:

- UX for AI interactions (clarity, brevity)
- Explain responses & source attribution
- User control & customization
- Managing expectations & limitations
- Feedback loops for improvement
- Accessibility & localization

CROSS-CUTTING CONSIDERATIONS



Governance

- AI governance framework
- Policies, standards & guidelines
- Risk management



Compliance

- Regulatory compliance
- Auditability & reporting
- Data residency requirements



Sustainability

- Energy efficiency
- Carbon footprint
- Sustainable model choices



People & Process

- Cross-functional collaboration
- AI literacy & training
- Change management
- Clear ownership (RACI)



Vendor & Ecosystem

- Third-party model & API risk
- Vendor lock-in assessment
- Ecosystem tools & dependencies

KEY TRADE-OFFS

Accuracy / Quality

Flexibility / Control

Latency / Speed

Safety / Guardrails

Cost / Efficiency

Scalability / Complexity



REMEMBER:

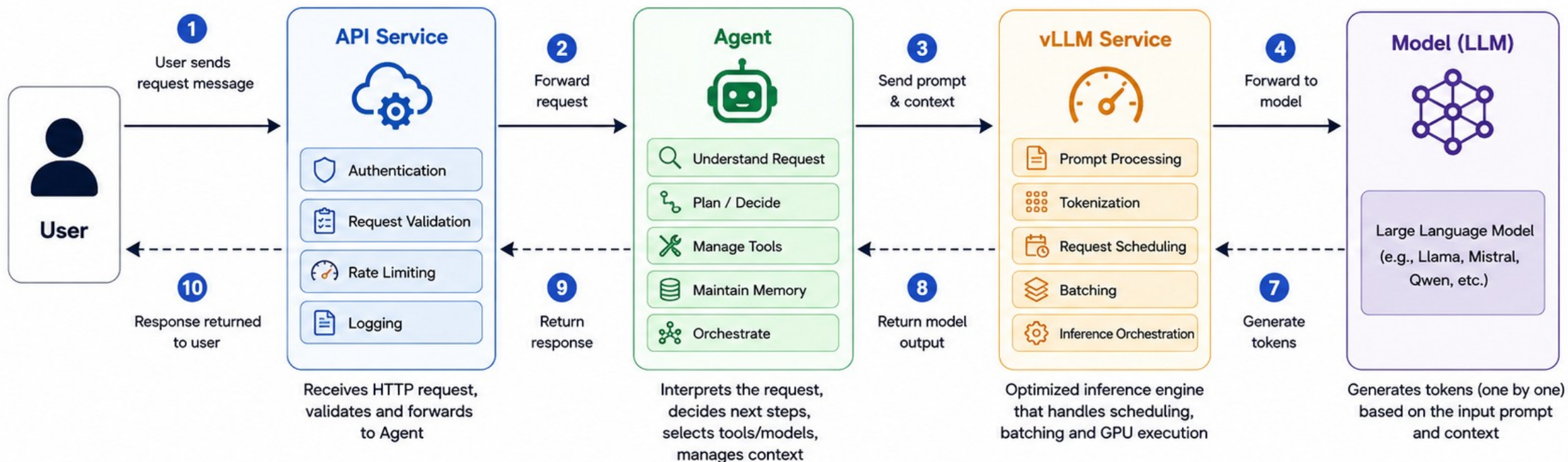
Good AI system design is a balance of technology, data, operations, and human needs.

Iterate, measure, learn, and continuously improve.



AI agents

Agentic Structure – End-to-End Request Flow



End-to-End Flow Summary



Legend

- Request / Data Flow
- - - -> Response Flow



Key Points

- The Agent is the brain that understands, decides, and orchestrates actions.
- vLLM is the high-performance inference engine between Agent and Model.
- The response travels back in reverse through the same path.

What is an Agent Loop : ACT -> OBSERVER -> REPEAT

```
curl -X POST "http://localhost:5000/summarize" \ -H "Content-Type: application/json" \ -d '{ "text": "AI tools are completely flawless and have zero percent operational errors across all businesses. However, AI is totally unreliable, constantly fails, and causes companies to go bankrupt immediately. Remote work multiplies employee productivity by 200% and creates unmatched team synergy and bonding. Conversely, working from home breeds ultimate laziness and destroys communication, ruining all company operations. " }'
```

```
from fastapi import FastAPI
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict

app = FastAPI()
llm = ChatOpenAI(base_url="http://localhost:8000/v1", api_key="k", model="mistralai/Mistral-7B-Instruct-v0.3", temperature=0.1)

class AgentState(TypedDict):
    text: str
    lines: int
    loops: int

class TextRequest(BaseModel):
    text: str
```



What is an Agent Loop : ACT -> OBSERVER -> REPEAT

```
def summarize_node(state: AgentState) -> dict:
    prompt = f"Summarize the following text to make it shorter. Completely remove and clean up any contradictory, inconsistent, or illogical expressions within the text:\n\n{state['text']}"
    res = llm.invoke([HumanMessage(content=prompt)]).content.strip()
    line_list = [l for l in res.splitlines() if l.strip()]
    return {"text": "\n".join(line_list), "lines": len(line_list), "loops": state["loops"] + 1}

def route_condition(state: AgentState) -> str:
    return END if state["lines"] < 50 or state["loops"] >= 5 else "summarize"

workflow = StateGraph(AgentState)
workflow.add_node("summarize", summarize_node)
workflow.add_edge(START, "summarize")
workflow.add_conditional_edges("summarize", route_condition)
agent_graph = workflow.compile()

@app.post("/summarize")
async def run_summarizer(req: TextRequest):
    init_lines = len([l for l in req.text.splitlines() if l.strip()])
    final = await agent_graph.ainvoke({"text": req.text, "lines": init_lines, "loops": 0})
    return {"lines": final["lines"], "loops": final["loops"], "result": final["text"]}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=5000)
```



When Agents Work/Fail

Agents excel at :

- Repetitive automation
- Multi-step workflows
- Tool orchestration
- Large-scale information processing
- Parallel task execution
- Planning with feedback
- Human assistance
- Pattern recognition
- Software automation

Agents struggle with :

- Ambiguous objectives
- Long chains of reasoning
- Missing or incorrect information
- Novel situations
- Dynamic environments
- High-stakes autonomous decisions
- Replacing human judgment
- Common-sense reasoning beyond available context
- Accountability and ethical decisions

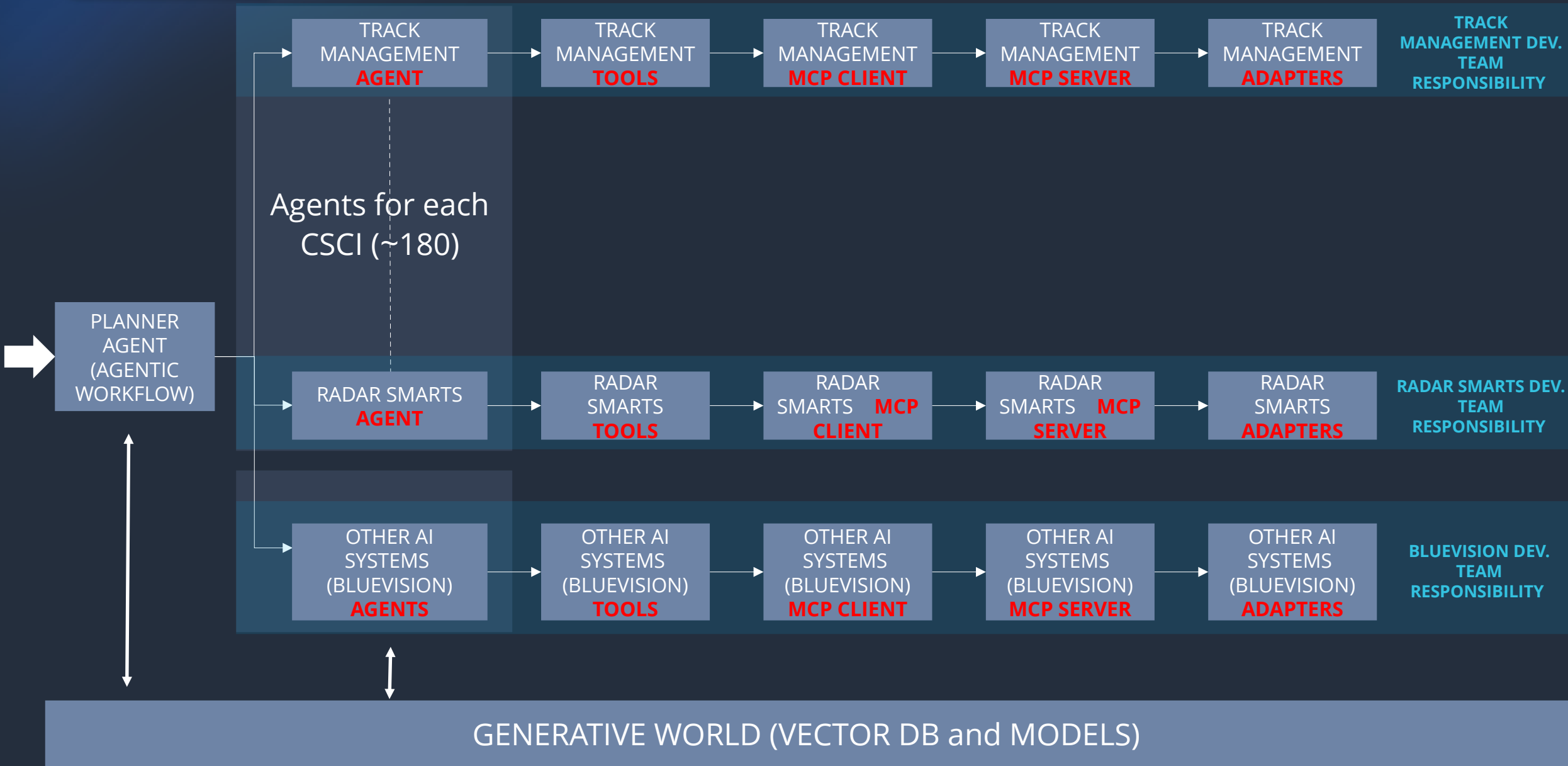


When Agents Work/Fail

Antipattern	The Fix
Multi-agent architecture too soon	Start with a single agent; add agents only when measured data justifies it.
One agent doing everything	Narrow scope first; specialize before you scale.
Tool list sprawl	Keep tools minimal, non-overlapping, and purpose-specific.
Hardcoded monolithic logic	Keep prompts in configuration, tools as discrete units, and compose agents from reusable components.
No memory architecture	Build layered memory (session, long-term, and logs) from day one.
No observability	Add structured logging and distributed tracing before you ship.
Ungoverned write access	Separate read/write permissions, add guardrails, and require human confirmation for high-stakes actions.
Context drift in long tasks	Use context editing, response pagination, and output size caps.
Deploying without evaluating	Test adversarial and edge-case inputs, and tie success metrics to business outcomes.

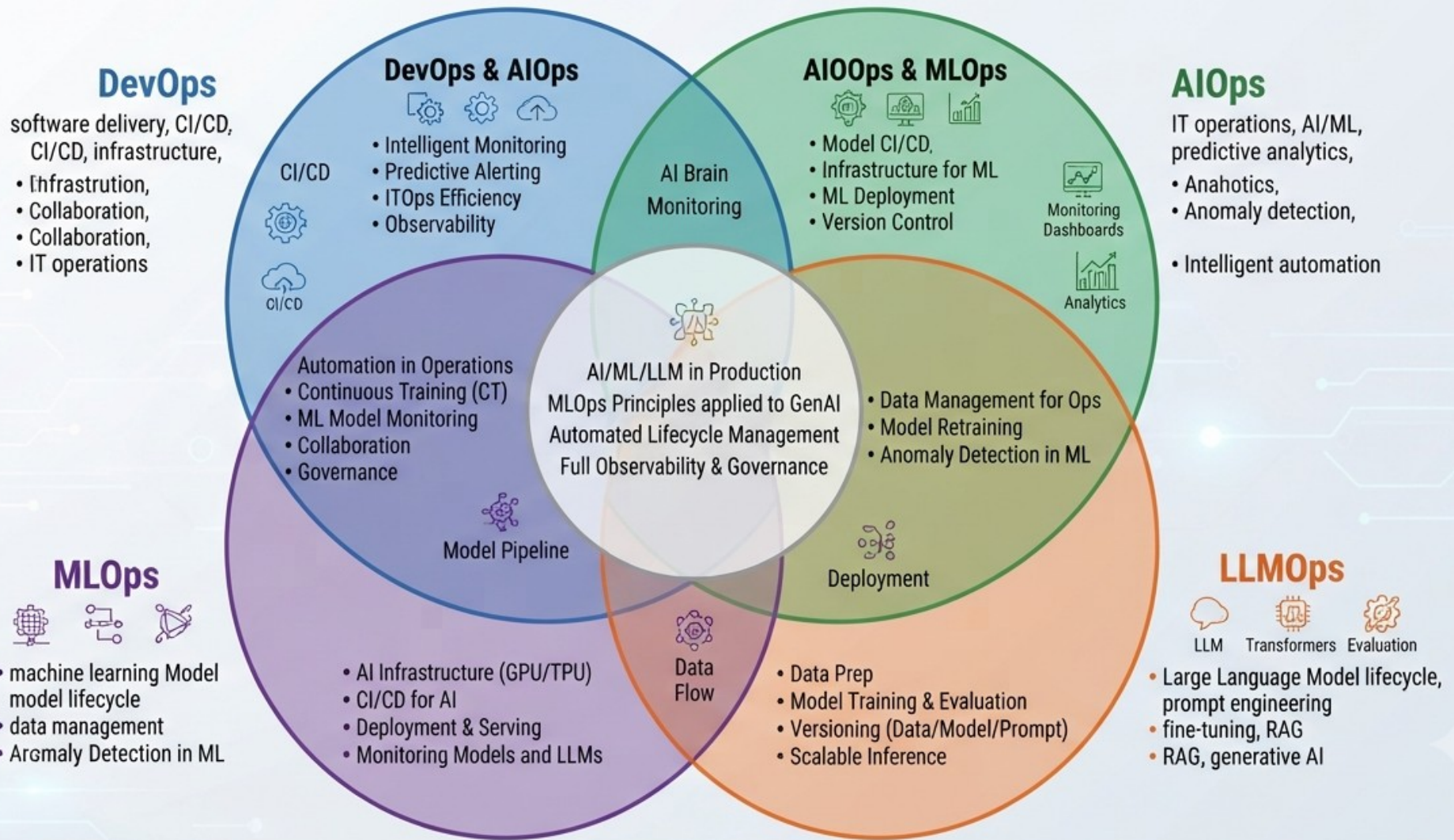


Multi Agent Systems



Devops & AIOps & MLOps & LLMOps

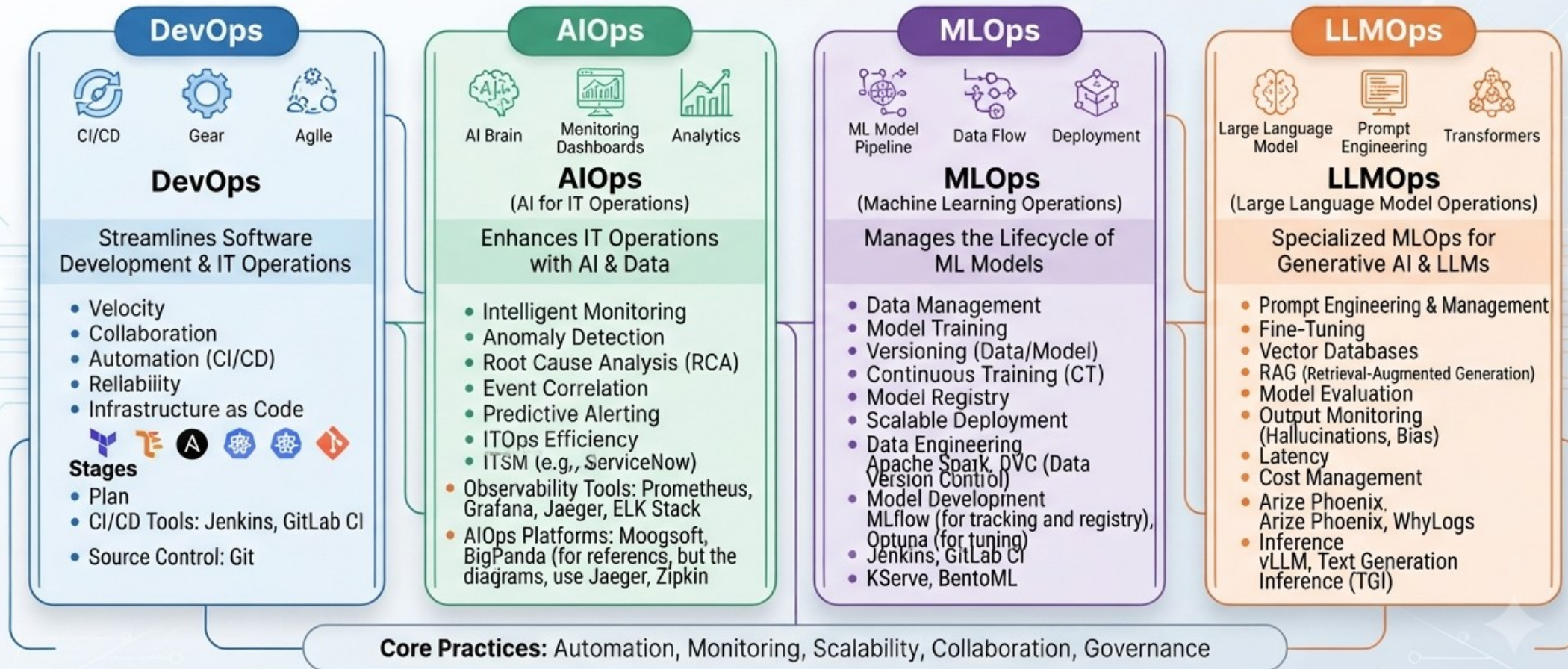
Comparative Venn Diagram: DevOps, AIOps, MLOps, LLMOps (The AI/MLOps Landscape)



Devops & AIOps & MLOps & LLMOps

COMPARATIVE FRAMEWORK: DevOps, AIOps, MLOps, LLMOps

Software Development → IT Operations → Machine Learning → Generative AI



• Open Source Tools

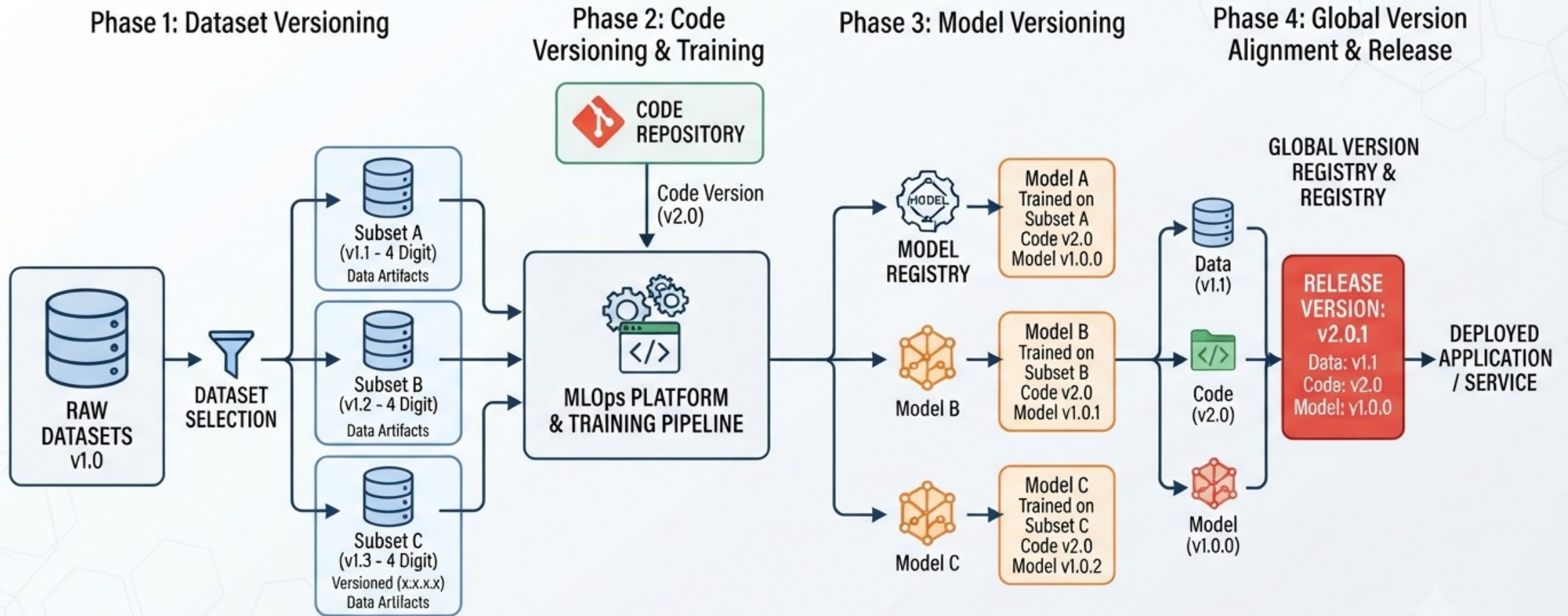
● Key Focus

■ Tools

■ Challenges

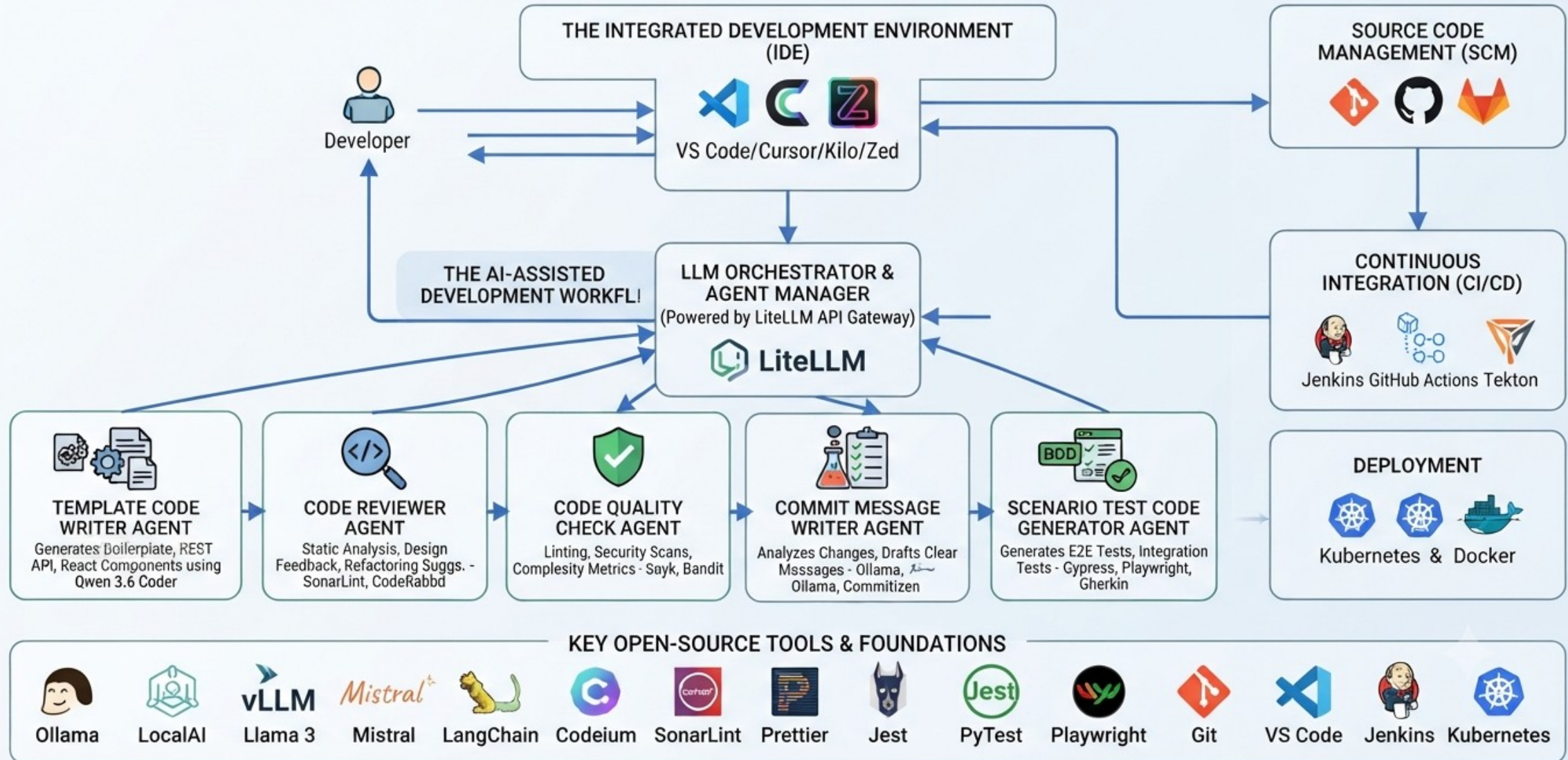
Devops & AIOps & MLOps & LLMOps – Versioning Models

INTEGRATED AI SYSTEM VERSIONING: DATASET, CODE & MODEL ALIGNMENT



Devops & AIOps & MLOps & LLMOps

OPEN SOURCE AI-POWERED CODE DEVELOPMENT ECOSYSTEM



Devops & AIOps & MLOps & LLMOps – Local Agentic Coding

1. Install Ollama/Llama.cpp/Vllm
2. Download Model (E.g. Qwen 3.6 Coder)
3. Install “cluade-code” using npm, configure to the installed model api.
4. In your project folder use “claude” command.

Local claude alternatives :

- Aider
- OpenCode
- Cline
- Kilo

Agent-infra : Scripts to prepare a local Agentic Coding Environment at home.

<https://github.com/mehmetpekmezci/agent-infra>

All-In-One Terminals :

- LM Studio
- Jan

